MULTI-ATTRIBUTE SKIN DISEASE DETECTION AND TREATMENT RECOMMENDATIONS USING DEEP LEARNING

MINI PROJECT REPORT

Submitted by

SHIBIA A

Register No: 720725207076

In partial fulfillment for the award of the degree

Of

MASTER OF COMPUTER APPLICATIONS

in

COMPUTER APPLICATIONS

HINDUSTHAN COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Valley Campus, Pollachi Highways,

Coimbatore – 641032

October 2026

HINDUSTHAN COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution, Affiliated to Anna University, Chennai)

COIMBATORE – 641 032

Department of Computer Applications

24CA2801 – MINI PROJECT

October 2026

This is to certify that the Project Work entitled

MULTI-ATTRIBUTE SKIN DISEASE DETECTION AND TREATMENT RECOMMENDATIONS USING DEEP LEARNING

is the bonafide record of Project work done by

SHIBIA A

Register No: 720725207076

of Master of Computer Applications during the year 2026 - 2027

Project Guide	**Head of the Department**

Submitted for the 24CA2801–Mini Project Viva-Voce Examination held on _____

Internal Examiner	**External Examiner**

DECLARATION

I affirm that the project work entitled **"MULTI-ATTRIBUTE SKIN DISEASE DETECTION AND TREATMENT RECOMMENDATIONS USING DEEP LEARNING"** being submitted in partial fulfillment for the award of Master of Computer Applications is the original work carried out by me. It has not formed the part of any other project work submitted for award of any degree or diploma, either in this or any other University.

(Signature of the Candidate)
SHIBIA A
Register No: 720725207076

I certify that the declaration made above by the candidate is true.

(Signature of the Guide)
Dr. K. VIGNESH KUMAR
Assistant Professor-MCA

ACKNOWLEDGEMENT

I take this opportunity to express my gratitude to everyone who helped me in this project. It is my honor and bounded duty to thank our Chairman **Shri.T.S.R.KHANNAIYANN,** Secretary **Mrs. SARASUWATHI KHANNAIYANN,** and Executive Secretary **Dr. K PRIYA** of Hindusthan College of Engineering and Technology for their academic interest shown towards the students.

I wish to express my deep sense of gratitude to our Principal Dr**. J.JAYA,** Hindusthan College of Engineering and Technology for providing us an opportunity to fulfill our project work.

I am greatly indebted to **Dr. A.R. JAYASUDHA,** Professor & Head, Department of Computer Applications (MCA) for having permitted us to carry out this Mini project work and giving the complete freedom to utilize the resources of the department.

I extend my whole hearted thanks to my Project Guide **Dr.K.VIGNESH KUMAR,** Assistant Professor, Department of Computer Applications (MCA) for rendering his valuable service throughout the tenure of the project.

I also convey my heart full thanks to all the teaching and non-teaching staff of the Department and friends for their valuable support which energized me to complete this project.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	Abstract	v
	List of Figures	vi
	List of Tables	vii
1	**INTRODUCTION**	
	1.1 Objective	1
	1.2 Scope of the Project	1
2	**SYSTEM ANALYSIS**	
	2.1 Existing System	2
	2.2 Proposed System	3
3	**SYSTEM SPECIFICATION**	
	3.1 Hardware Specification	4
	3.2 Software Specification	4
4	**SOFTWARE DESCRIPTION**	
	4.1 About Front End	5
	4.2 About Back End	6
	4.3 About Deep Learning Framework	7
5	**PROJECT DESCRIPTION**	
	5.1 System Architecture	8
	5.2 Data Flow Diagram	8
	5.3 Use Case Diagram	11
	5.4 ER Diagram	12
	5.5 Database Design	13
	5.6 Dataset Description	14
	5.7 Deep Learning Model Design	16
	5.8 Module Description	18
	5.9 Input Design	20
	5.10 Output Design	21

CHAPTER NO	TITLE	PAGE NO
6	**SYSTEM TESTING**	
	6.1 Testcase	24
	6.2 Model Performance Evaluation	26
7	**SYSTEM IMPLEMENTATION**	
	7.1 Introduction	30
	7.2 Implementation Process	30
8	**CONCLUSION AND FUTURE ENHANCEMENT**	31
9	**APPENDICES**	
	9.1 Source Code	33
	9.2 Screenshots	39
10	**REFERENCES**	43

ABSTRACT

Skin diseases are among the most common health problems in the world, and the early identification of skin cancers such as melanoma and basal cell carcinoma greatly improves the chances of successful treatment. However, expert dermatological examination is not always easily available, and visual diagnosis can be subjective. This project presents "Multi-Attribute Skin Disease Detection and Treatment Recommendations using Deep Learning", a web-based system that analyses a skin image uploaded by the user and produces several related outputs: the skin status (normal or diseased), the predicted disease class, a confidence score for the prediction, and supportive treatment information.

The system uses a two-stage ResNet18 pipeline implemented with PyTorch. Stage 1 separates normal skin from diseased skin. When a lesion is suspected, Stage 2 classifies the image into one of eight classes: Actinic Keratosis (AK), Basal Cell Carcinoma (BCC), Benign Keratosis-like Lesions (BKL), Dermatofibroma (DF), Melanoma (MEL), Melanocytic Nevus (NV), Squamous Cell Carcinoma (SCC) and Vascular Lesions (VASC). The disease classifier was developed using a labelled dataset of 25,319 lesion images and uses transfer learning, data augmentation and a class-balanced sampler to cope with the strong imbalance between classes. On a validation set of 5,067 images the model achieved an accuracy of 81.71%, a weighted precision of 82.70% and a weighted F1-score of 81.91%, while the Stage 1 normal/diseased detector reached a final validation accuracy of 99.50%.

The trained models are integrated into a Flask web application with an SQLite database. Users can register and log in securely with hashed passwords, upload a skin image, view the prediction with its confidence, and open a treatment page that provides medicinal plant information and general skin-care guidance together with an advisory to consult a qualified doctor. The system is intended for education and awareness and does not replace professional medical diagnosis.

LIST OF FIGURES

Figure No	Title	Page No
5.1	System Architecture Diagram	8
5.2	DFD Level 0 (Context Diagram)	9
5.3	DFD Level 1	10
5.4	DFD Level 2 (Prediction Process)	11
5.5	Use Case Diagram	12
5.6	Entity Relationship Diagram (ER Diagram)	13
5.7	Dataset Distribution	16
5.8	Two-Stage Prediction Pipeline	16
5.9	Model Training Workflow	17
5.10	Login Page (Input Form)	20
5.11	Skin Image Upload Page	21
5.12	Prediction Result - Diseased Skin (AK)	21
5.13	Prediction Result - Normal Skin	22
5.14	Treatment Recommendation Page	22
5.15	General Skin Care Section	23
6.1	Test Case TC02 - Login with Wrong Password	24
6.2	Test Case TC08 - Register with Invalid E-mail Format	25
6.3	Test Case TC10 - Submit without Selecting an Image	26
6.4	Test Case TC17 - View Profile	26
6.5	Confusion Matrix of the 8-Class Disease Model	27
6.6	Class-wise Precision, Recall and F1-score	28
9.1	Home Page	39
9.2	Login Page	39
9.3	Skin Image Upload Page	40
9.4	Prediction Result (AK)	40
9.5	Treatment Recommendation Option	41
9.6	Treatment Recommendation Page	41
9.7	General Skin Care and Advisory	42
9.8	Prediction Result (Normal Skin)	42

LIST OF TABLES

Table No	Title	Page No
3.1	Hardware Specification	4
3.2	Software Specification	4
4.1	Flask Routes of the Application	6
5.1	USERS	13
5.2	Prediction Result Structure (Runtime Output)	14
5.3	Skin Disease Classes Used in the Project	14
5.4	Dataset Distribution	15
5.5	Training Configuration of the 8-Class Model	18
6.1	LOGIN PAGE	24
6.2	REGISTRATION PAGE	25
6.3	IMAGE UPLOAD AND PREDICTION	25
6.4	TREATMENT, PROFILE AND SESSION	26
6.5	Overall Performance of the 8-Class Model	27
6.6	Class-wise Performance Metrics	28
6.7	Most Frequent Misclassifications	29
6.8	Stage 1 Normal Skin Detector Evaluation	29

CHAPTER 1

INTRODUCTION

1.1 OBJECTIVE

The main objective of this project is to develop an intelligent, web-based system that can analyse a photograph of the skin, decide whether it is normal or affected, identify the most probable disease among eight clinically important classes, report the model's confidence, and provide supportive treatment and skin-care information. Deep learning for image analysis is combined with a Flask web application so that the trained models can be used easily through a web browser.

Skin diseases range from harmless benign growths to dangerous cancers such as melanoma. Many people ignore early skin changes, so a quick image-based first opinion can encourage earlier medical advice. The system is "multi-attribute" because one image upload produces the skin status, disease class, prediction confidence and treatment recommendation.

SPECIFIC OBJECTIVES

- To build a two-stage ResNet18 pipeline: Stage 1 separates normal from diseased skin and Stage 2 classifies the disease.
- To classify skin lesions into eight classes: Actinic Keratosis (AK), Basal Cell Carcinoma (BCC), Benign Keratosis-like Lesions (BKL), Dermatofibroma (DF), Melanoma (MEL), Melanocytic Nevus (NV), Squamous Cell Carcinoma (SCC) and Vascular Lesions (VASC).
- To handle the severe class imbalance (12,875 nevus against 239 dermatofibroma images) using transfer learning, augmentation and a weighted sampler.
- To evaluate the model using accuracy, precision, recall, F1-score and a confusion matrix.
- To provide multi-attribute output: skin status, disease, confidence and treatment information in one result.
- To implement secure user registration and login with hashed passwords and session-based access control, through a simple and responsive web interface.
- To provide medicinal plant information and skin-care guidance with a medical-consultation advisory, encouraging early consultation with a dermatologist.

1.2 SCOPE OF THE PROJECT

The system accepts a clear photograph of a skin region and processes it through two trained neural networks covering one normal-skin category and eight disease categories. The treatment page offers educational information on medicinal plants and skin care. The scope is limited to image-based screening for these classes; it does not use patient history, does not prescribe medicines, and its results must always be confirmed by a doctor.

CHAPTER 2

SYSTEM ANALYSIS

2.1 EXISTING SYSTEM

In the traditional approach, a skin condition is examined visually by a dermatologist, often with the help of a dermatoscope. When a lesion appears suspicious, a skin biopsy is taken and examined under a microscope. This process is accurate when performed by an expert, but it depends on the availability of specialists, on time and on cost. In many towns and villages, a dermatologist may not be easily reachable, and people often rely on guesses, home remedies or general internet searches.

Several computer-aided tools and mobile applications also exist, but many of them are limited. Some only separate benign from malignant lesions, some work on very few disease types, and few of them first check whether the skin is normal at all. Most do not show the model confidence, and most do not connect the result to any supportive information for the user.

Limitations of the Existing System

- Expert dermatological examination is not available to everyone, especially in rural and remote areas.
- Visual diagnosis is subjective and may differ from one examiner to another.
- Consultation, dermoscopy and biopsy take time and are costly for routine screening.
- People often delay seeing a doctor because they cannot tell whether a skin change is serious.
- Many automated tools handle only a few classes or only a benign/malignant decision.
- Many tools assume that every uploaded image is diseased and do not check for normal skin.
- Datasets are highly imbalanced, so simple models become biased towards the most common class.
- Confidence of the prediction and supportive treatment information are usually not provided together.

Because of these limitations, there is a need for an accessible, web-based screening assistant that can quickly analyse an image, handle normal skin and multiple diseases, report its confidence and give basic guidance.

2.2 PROPOSED SYSTEM

The proposed system is a web-based application that uses a two-stage deep learning pipeline. After the user logs in and uploads a skin image, the image is resized and normalised. Stage 1, a ResNet18 binary classifier, decides whether the skin is normal or diseased. If the skin is normal, the system reports “Normal Skin” with its confidence. If a disease is suspected, Stage 2, a second ResNet18 classifier, identifies the most probable

disease from eight classes and reports the confidence. The user can then open a treatment page that presents medicinal plant information and general skin-care guidance with a medical advisory.

Key Features of the Proposed System

- Two-stage prediction: normal/diseased screening followed by 8-class disease classification.
- Multi-attribute output: skin status, disease name, confidence percentage and treatment information.
- Transfer learning with a pretrained ResNet18 and fine-tuning on skin lesion images.
- Class-balanced training using a weighted random sampler and image augmentation.
- Secure registration and login with hashed passwords and session management.
- Display of the uploaded image together with the prediction result.
- Treatment page with medicinal plant information, general skin-care tips and a medical-consultation advisory.
- User profile page showing account information.

Advantages of the Proposed System

- Gives a quick first screening result using only a photograph and a web browser.
- Checks for normal skin first, which avoids forcing a disease label on healthy skin.
- Shows how confident the model is, helping users judge the result carefully.
- Improves performance on rare classes through balanced sampling.
- Combines prediction and supportive information in a single, simple workflow.
- Encourages early medical consultation and increases awareness of skin health.

The proposed system provides a smooth, secure and understandable screening experience while making it clear that the output is educational and not a final diagnosis.

CHAPTER 3

SYSTEM SPECIFICATION

3.1 HARDWARE SPECIFICATION

Table 3.1 Hardware Specification

Hardware	Description / Purpose
Processor	Intel Core i5 / AMD Ryzen 5 or higher for model training and smooth execution (Core i3 is sufficient for running the trained models)
RAM	Minimum 8 GB (16 GB recommended for training on the full image dataset)
Storage	Minimum 10 GB free HDD / SSD for the dataset, model weight files, uploaded images and dependencies
Graphics Card (Optional)	NVIDIA GPU with CUDA support to speed up training; the application automatically uses the CPU when no GPU is available
System Architecture	64-bit system required for Python, PyTorch and modern development tools

3.2 SOFTWARE SPECIFICATION

Table 3.2 Software Specification

Software	Description / Purpose
Operating System	Windows 10 / 11, Linux or macOS
Programming Language	Python 3.9 or above was installed, with Flask, Werkzeug, PyTorch, torchvision, Pillow, scikit-learn and NumPy (using pip). Visual Studio Code was used as the development environment.
Web Framework	Flask with Werkzeug for routing, sessions, file handling and password hashing
Deep Learning Framework	PyTorch and torchvision for ResNet18, image transformations and model loading
Supporting Libraries	Pillow for image reading, scikit-learn for data splitting and F1-score, NumPy
Database	SQLite (sqlite3 module) for storing registered user accounts
Front-End Technologies	HTML5, CSS3, Jinja2 templates and JavaScript for the user interface
Development Tool	Visual Studio Code
Web Browser	Google Chrome, Mozilla Firefox or Microsoft Edge for accessing the system

CHAPTER 4

SOFTWARE DESCRIPTION

4.1 ABOUT FRONT END

The front end of the system is built with HTML5 and CSS3 pages that are rendered on the server by the Jinja2 templating engine of Flask. Each page is a template: the landing page (index.html), registration (register.html), login (login.html), image upload (predict.html), prediction result (result.html), treatment recommendation (treatment.html) and user profile (profile.html). Dynamic values such as the user name, the predicted disease, the confidence and the uploaded image name are inserted into the templates by Flask at run time. Styling is written in plain CSS, and a small amount of JavaScript is used for interface behaviour such as showing and hiding the password on the login page. No external front-end framework is required, which keeps the pages light and easy to run offline.

TECHNOLOGIES USED

- **HTML5:** Defines the structure of all pages, including the registration form, login form and the multipart image-upload form with an image file input.
- **CSS3:** Controls layout, colours, cards, buttons, navigation bars and responsive behaviour so that the pages can be used on desktops, laptops, tablets and mobile phones.
- **Jinja2 Templates:** Insert dynamic data into pages, apply conditions (for example, hiding the treatment button for normal skin) and generate links using url_for().
- **JavaScript:** Provides small interactive features on the client side such as password visibility toggling.

FRONT-END FEATURES

- Responsive pages with a clean, modern layout and consistent colours.
- Landing page explaining the system, how it works and the eight supported disease classes.
- Registration and login forms with required-field and e-mail format validation.
- Image upload page that accepts image files and shows a clear “Detect Skin Disease” button.
- Result page showing the uploaded image, the prediction and the confidence percentage.
- Treatment page with medicinal plant cards, general skin-care cards and an important medical advisory.
- Profile page showing the logged-in user’s name and e-mail, and a navigation bar with Home, Detection, Profile and Logout.

OBJECTIVE OF THE FRONT END

The objective of the front end is to give users a simple and attractive interface through which they can register, log in, upload a skin image, understand the prediction and read supportive information without any technical knowledge. The interface also shows warnings and disclaimers so that users understand that the system is an educational tool.

4.2 ABOUT BACK END

The back end is written in Python using the Flask micro-framework. The file app.py is the entry point of the application. It creates the Flask app, defines the upload folder and database path, creates the users table if it does not exist, and defines all routes. The file prediction.py contains the deep learning inference code: it loads the two trained ResNet18 models once when the server starts and exposes the function predict_image(), which is called by the /predict route. The file train_resnet18_improved.py is used offline to train the 8-class disease model. Trained weights are stored in .pth files and the class names are stored in class_names.json.

TECHNOLOGIES USED

- **Python and Flask:** Flask handles HTTP requests, routing, template rendering, file upload and the login session. The secret key is used to sign the session cookie.
- **SQLite:** A lightweight file-based database (users.db) stores registered users with a unique username and a unique e-mail.
- **Werkzeug Security:** generate_password_hash() stores only a hashed password, and check_password_hash() verifies the password at login. secure_filename() cleans the names of uploaded files.
- **Authentication and Authorization:** After a successful login the username is stored in the session. The prediction, treatment and profile routes redirect to the login page when no user is logged in.
- **Prediction Module:** Converts the uploaded image to RGB, resizes it to 224 × 224, normalises it and passes it through the two ResNet18 models to obtain the result.

Table 4.1 Flask Routes of the Application

Route	Method	Login Needed	Description
/	GET	No	Landing (home) page of the system
/register	GET, POST	No	Shows the registration form and creates a new user with a hashed password
/login	GET, POST	No	Verifies the credentials and starts the session; redirects to the prediction page
/logout	GET	Yes	Clears the session and returns to the home page

Route	Method	Login Needed	Description
/predict	GET, POST	Yes	Shows the upload page; on POST saves the image, runs the two-stage prediction and shows the result
/uploads/<filename>	GET	No	Serves an uploaded image so that it can be displayed on the result page
/treatment/<disease>	GET	Yes	Shows medicinal plant information and general skin-care guidance
/profile	GET	Yes	Shows the username and e-mail of the logged-in user

4.3 ABOUT DEEP LEARNING FRAMEWORK

The deep learning part of the project is implemented with PyTorch and torchvision. PyTorch provides tensors, automatic differentiation, optimisers and GPU support, while torchvision provides the ImageFolder dataset class, image transformations and pretrained model architectures.

ResNet18

ResNet18 is an 18-layer convolutional neural network from the Residual Network family. Its main idea is the residual (skip) connection, in which the input of a block is added to the output of its convolution layers. These shortcuts make deep networks easier to train and reduce the vanishing-gradient problem. ResNet18 has about 11 million parameters and ends with global average pooling and a fully connected layer. In this project the final fully connected layer is replaced: it has 2 outputs in Stage 1 (normal and diseased) and 8 outputs in Stage 2 (the disease classes).

Transfer Learning

Instead of training from random weights, the 8-class model starts from ResNet18 weights pretrained on the large ImageNet dataset. These weights already detect edges, textures and shapes. Fine-tuning them on skin images gives better accuracy with less training time and less data than training from scratch.

Other Libraries

- **Pillow:** Opens the uploaded image and converts it to RGB.
- **scikit-learn:** Performs the stratified train/validation split and calculates the macro F1-score.
- **Softmax:** Converts the network outputs into class probabilities; the highest probability is shown as the confidence.

CHAPTER 5

PROJECT DESCRIPTION

5.1 SYSTEM ARCHITECTURE

The architecture of the system has four layers. The presentation layer is the web browser, which shows the HTML pages and sends requests. The application layer is the Flask server (app.py), which handles routing, sessions, file upload and calls the prediction module. The deep learning layer contains the two ResNet18 models, loaded with PyTorch when the server starts. The data layer consists of the SQLite database for user accounts and the uploads folder for submitted images.

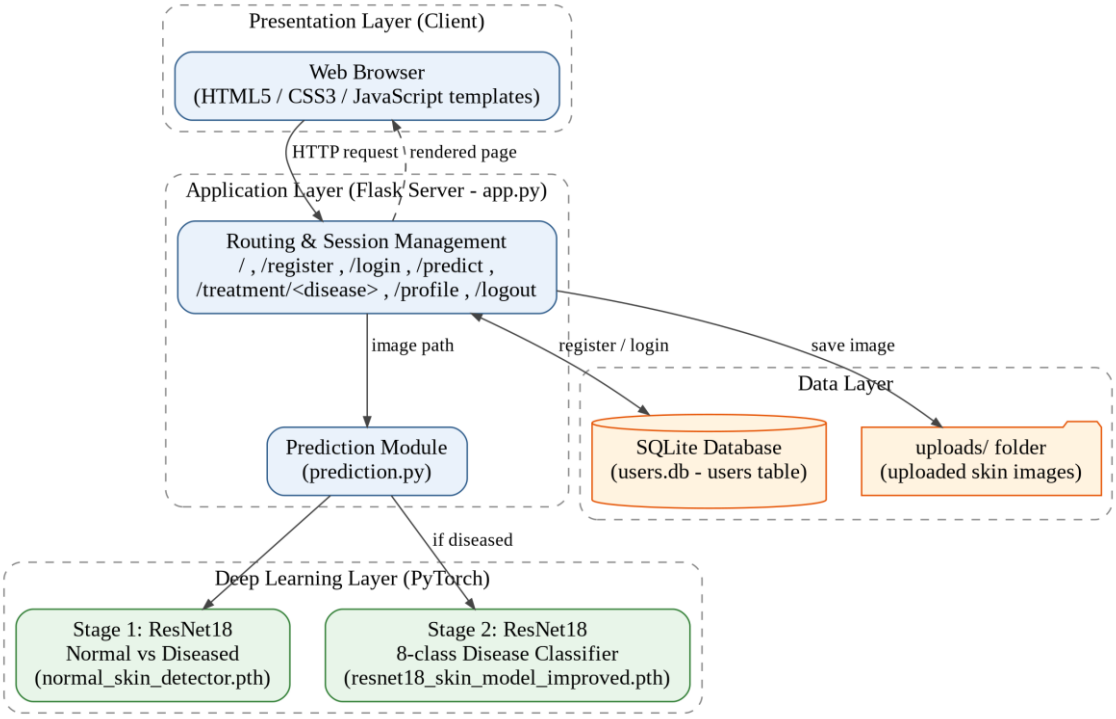


Figure 5.1 System Architecture Diagram

When the user uploads an image, Flask saves it in the uploads folder and sends its path to predict_image(). The function applies the Stage 1 model; if the skin is normal, the result is returned immediately. Otherwise the same image tensor is passed to the Stage 2 model and the disease class and confidence are returned to Flask, which renders the result page.

5.2 DATA FLOW DIAGRAM

The data flow diagrams show how data moves between the user, the processes of the system and the data stores. The system has a single external entity, the User, because the same login is used by everyone and the application does not contain a separate administrator module.

User Authentication & Authorization Module

- **User Registration & Login:** Provides registration and login using a username, e-mail and password; passwords are stored only as hashes.
- **Session Management:** Keeps the user logged in and redirects unauthenticated users to the login page when they try to open protected pages.
- **Access Control:** The prediction, treatment and profile pages are accessible only after login.

Level 0 (Context Diagram) shows the whole system as one process exchanging data with the user.

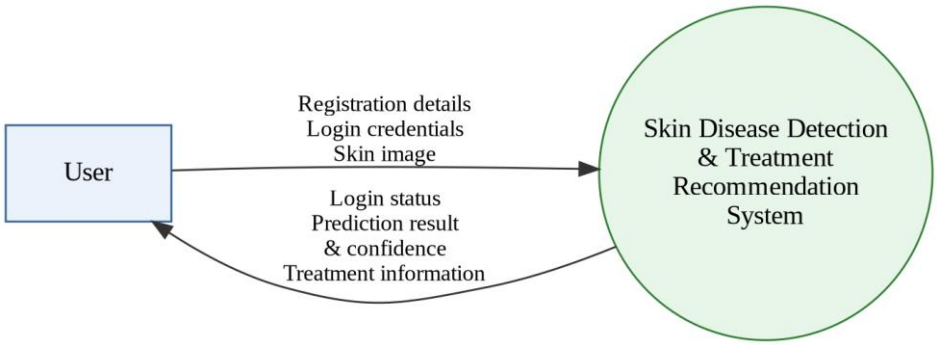


Figure 5.2 DFD Level 0 (Context Diagram)

Level 1 expands the system into five main processes: registration and login, image upload and preprocessing, Stage 1 normal/diseased detection, Stage 2 disease classification and treatment recommendation. The users table (D1) and the uploads folder (D2) are the data stores.

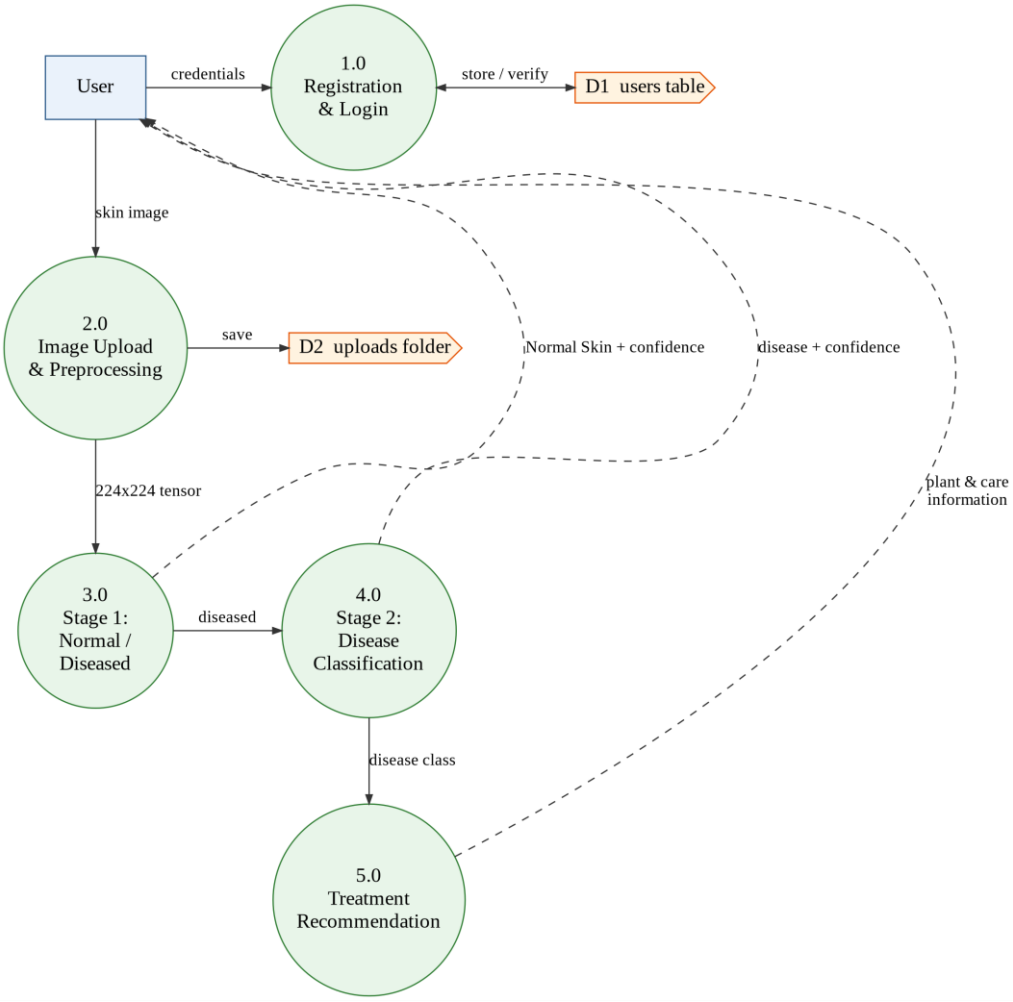


Figure 5.3 DFD Level 1

Level 2 expands the prediction process: the uploaded file is validated and saved, resized and normalised, passed through Stage 1, and, if the skin is diseased, through Stage 2. The result and treatment page are then built and shown to the user.

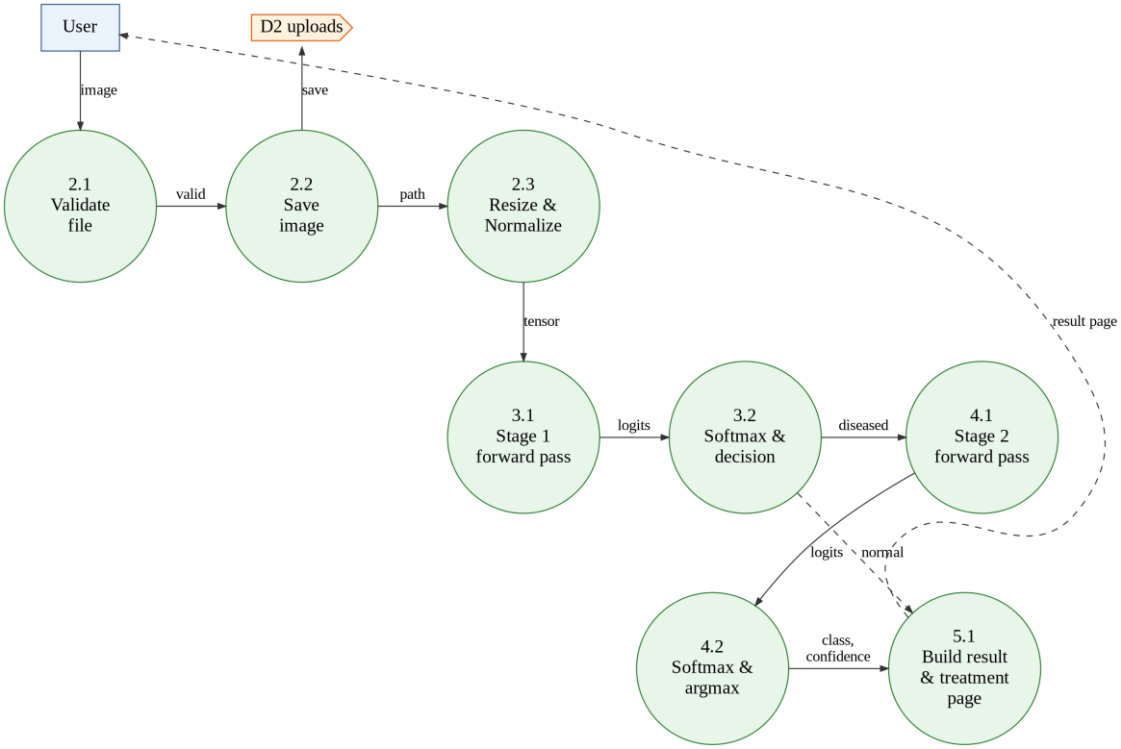


Figure 5.4 DFD Level 2 (Prediction Process)

5.3 USE CASE DIAGRAM

The use case diagram shows the functions that the user can perform. A new user registers and logs in. After login the user can upload a skin image, view the normal/diseased result with disease name and confidence, open the treatment recommendation page, view the profile and log out.

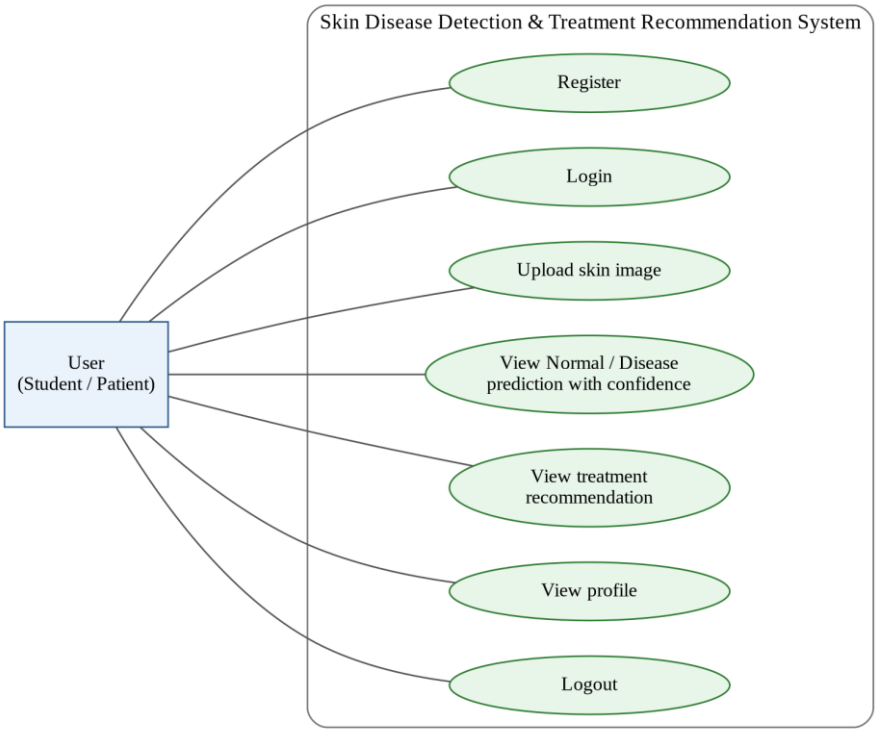


Figure 5.5 Use Case Diagram

5.4 ER DIAGRAM

The Entity Relationship diagram shows the main entities of the system and how they are related. Only the USERS entity is stored permanently in the SQLite database. The other entities describe the information that is handled at run time: the uploaded image is stored as a file in the uploads folder, the prediction is produced for each request and shown on the result page, the disease class comes from the file class_names.json and the treatment information is part of the treatment page. They are shown here to explain the logical design of the system.

Entities and Their Attributes

- **USERS:** id (PK), username, email, password (hashed).
- **SKIN_IMAGE:** filename (PK), file path in the uploads folder.
- **PREDICTION:** status (normal / diseased), class, confidence, stage1_confidence.
- **DISEASE_CLASS:** code (PK) – AK, BCC, BKL, DF, MEL, NV, SCC, VASC.
- **TREATMENT_INFO:** medicinal plants, general skin-care tips and medical-consultation advice.

Logical Relationships and Cardinality

- **USERS to SKIN_IMAGE (1 : N):** One user can upload many images; each uploaded image belongs to one user session.
- **SKIN_IMAGE to PREDICTION (1 : 1):** Each uploaded image is analysed into one prediction result.

- **DISEASE_CLASS to PREDICTION (1 : N):** One disease class can be the result of many predictions, and each diseased prediction has exactly one class.
- **DISEASE_CLASS to TREATMENT_INFO (1 : 1):** The detected disease class opens the treatment page. In the present version the same set of plant and skin-care information is shown for every class.

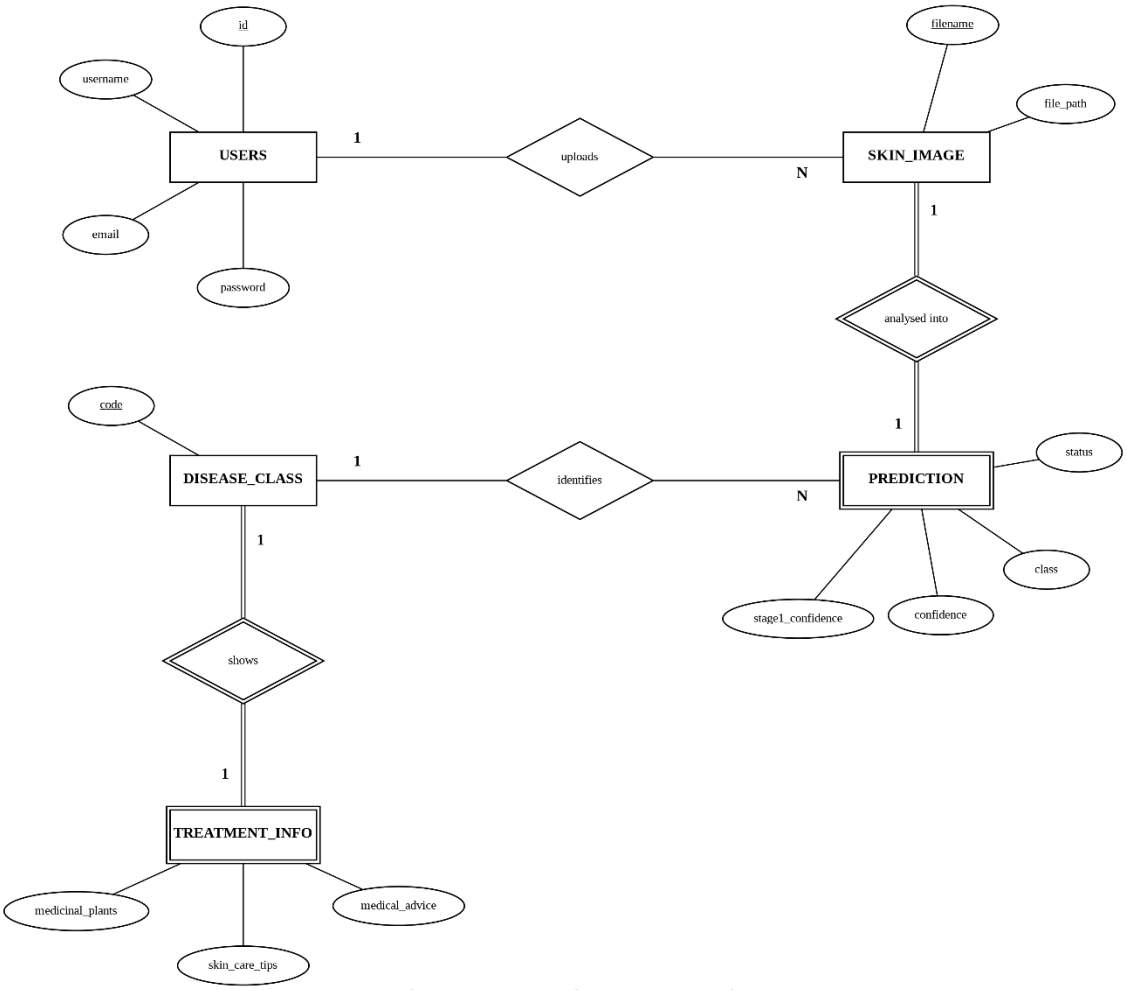


Figure 5.6 Entity Relationship Diagram (ER Diagram)

5.5 DATABASE DESIGN

The application uses SQLite, a file-based relational database stored in users.db. The users table is created automatically by the init_db() function when the application starts. The table stores account information; the password is never stored in plain text.

Primary key: id **Foreign key:** None

Table 5.1 USERS

Field	Type	Description
id	INTEGER (PK, AUTOINCREMENT)	Unique identifier of the user
username	TEXT (UNIQUE, NOT NULL)	Login name of the user
email	TEXT (UNIQUE, NOT NULL)	E-mail address of the user
password	TEXT (NOT NULL)	Password stored as a Werkzeug hash

Uploaded images are kept in the uploads folder. The prediction result is not stored in the database; it is created for every request and passed to the result page. Its structure is shown below.

Table 5.2 Prediction Result Structure (Runtime Output)

Field	Normal Skin Result	Diseased Result
status	"normal"	"diseased"
class	"Normal Skin"	One of AK, BCC, BKL, DF, MEL, NV, SCC, VASC
confidence	Stage 1 confidence (%)	Stage 2 confidence (%)
stage1_confidence	Same as confidence	Stage 1 confidence (%) that the skin is diseased

5.6 DATASET DESCRIPTION

The disease classifier was developed with a labelled skin lesion image dataset of eight classes containing 25,319 images; the class names and image counts follow the ISIC 2019 skin lesion classification challenge. The collection also includes 499 normal skin images, so the dataset holds 25,818 images in total. The disease images are arranged in one folder per class so that they can be read with the torchvision ImageFolder class. The Stage 1 normal/diseased detector was trained on a set of 1,000 images labelled Normal or Diseased (Section 5.7).

The normal skin images were collected from publicly available images through Google Images and used to create the normal skin dataset. Because these images come from public web sources, their lighting, resolution and skin tones vary, and they have not been clinically verified.

The Stage 1 dataset contains 1,000 images, consisting of 500 Normal skin images and 500 Diseased skin images.

Skin Disease Classes

Table 5.3 Skin Disease Classes Used in the Project

Code	Disease	Nature
AK	Actinic Keratosis	Rough, scaly patches caused by long-term sun exposure; considered pre-cancerous
BCC	Basal Cell Carcinoma	Most common skin cancer; slow growing and malignant
BKL	Benign Keratosis-like Lesions	Non-cancerous keratotic lesions such as seborrheic keratosis and solar lentigo
DF	Dermatofibroma	Common benign fibrous skin nodule
MEL	Melanoma	Dangerous malignant tumour of pigment cells; early detection is vital
NV	Melanocytic Nevus	Common benign mole
SCC	Squamous Cell Carcinoma	Malignant skin cancer arising from squamous cells
VASC	Vascular Lesions	Lesions of blood vessels such as angiomas; usually benign

Class Distribution

The dataset is strongly imbalanced. Melanocytic nevus alone forms about half of all images, while dermatofibroma and vascular lesions have only a few hundred images each. Training a model directly on such data would make it favour the common class, so the training process uses a balanced sampler and augmentation (Section 5.7).

Table 5.4 Dataset Distribution

S.No	Class	Number of Images	Share of Total
1	Actinic Keratosis (AK)	867	3.36%
2	Basal Cell Carcinoma (BCC)	3,322	12.87%
3	Benign Keratosis-like Lesions (BKL)	2,613	10.12%
4	Dermatofibroma (DF)	239	0.93%
5	Melanocytic Nevus (NV)	12,875	49.87%
6	Melanoma (MEL)	4,522	17.51%
7	Squamous Cell Carcinoma (SCC)	628	2.43%
8	Vascular Lesions (VASC)	253	0.98%
9	Normal Skin	499	1.93%
	Total	**25,818**	**100%**

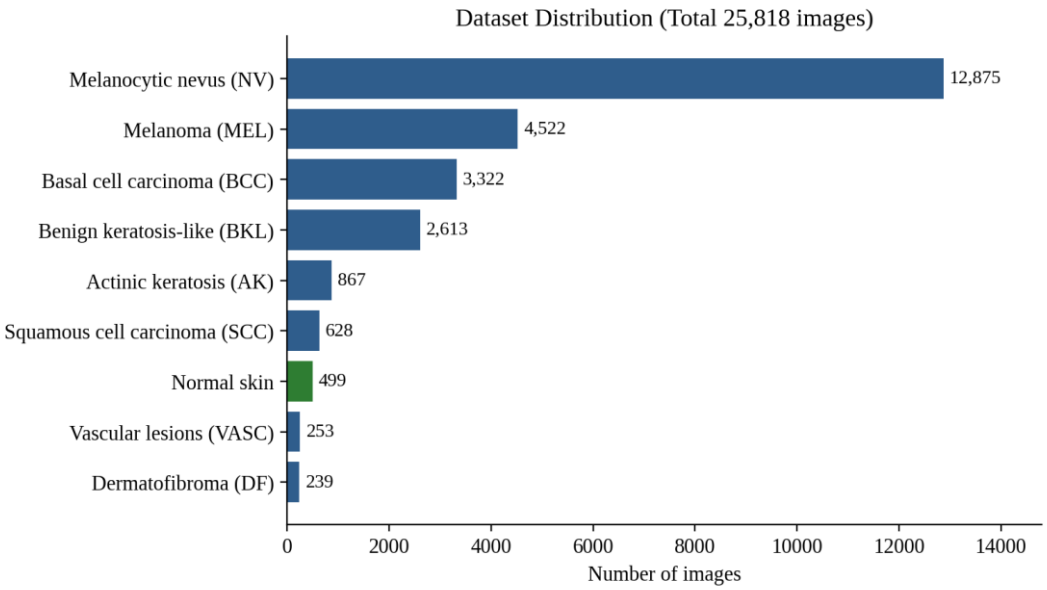


Figure 5.7 Dataset Distribution

5.7 DEEP LEARNING MODEL DESIGN

Two-Stage Pipeline

The system does not send every image directly to the disease classifier. A first model checks whether the skin is normal. Only images that are considered diseased go to the second model. This reduces the chance of giving a disease label to healthy skin.

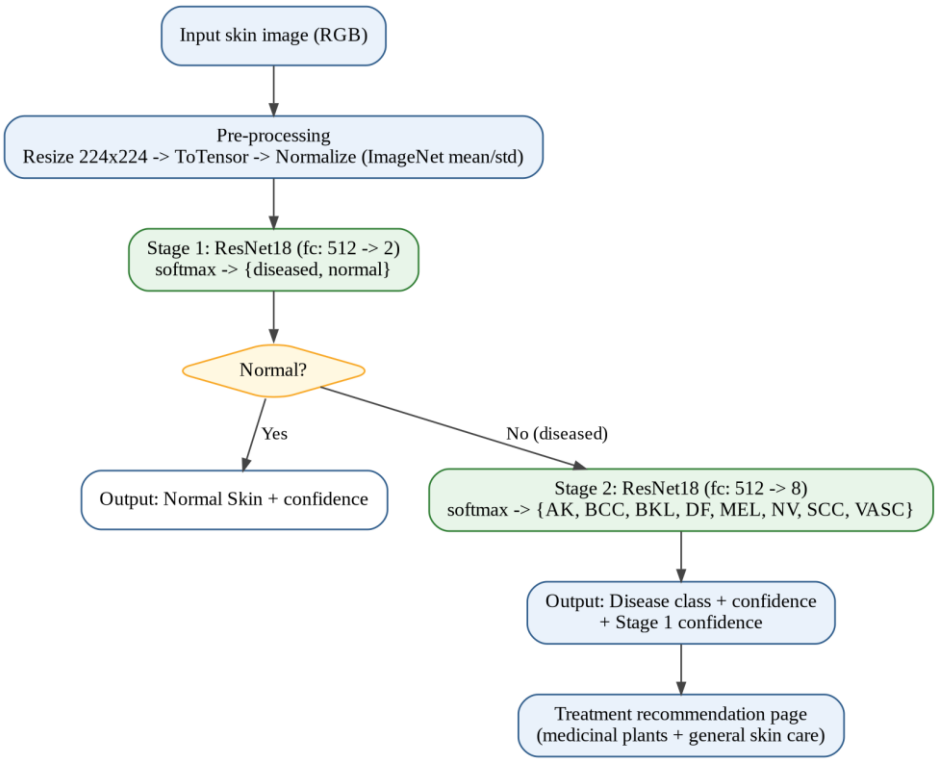


Figure 5.8 Two-Stage Prediction Pipeline

- **Pre-processing:** The image is converted to RGB, resized to 224 × 224 pixels, converted to a tensor and normalised with the ImageNet mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225).
- **Stage 1 (Normal / Diseased):** A ResNet18 whose final layer has two outputs. The class order is diseased = 0 and normal = 1. If the highest probability belongs to "normal", the result is Normal Skin.
- **Stage 2 (Disease Classification):** A ResNet18 whose final layer has eight outputs, in the order of class_names.json: AK, BCC, BKL, DF, MEL, NV, SCC, VASC. The class with the highest softmax probability is the predicted disease and its probability is the confidence.

Training of the 8-Class Model

The 8-class model is trained by the script train_resnet18_improved.py. The dataset is split into 80% training and 20% validation images using a stratified split, so that every class appears in both parts in the same proportion. A WeightedRandomSampler gives each training image a weight equal to the inverse of its class frequency, so that every batch contains a balanced mixture of classes. The training images are augmented by flips, rotation and colour changes. After each epoch the model is evaluated on the validation set and the weights are saved whenever the macro F1-score improves, so the final saved model is the best epoch rather than simply the last one.

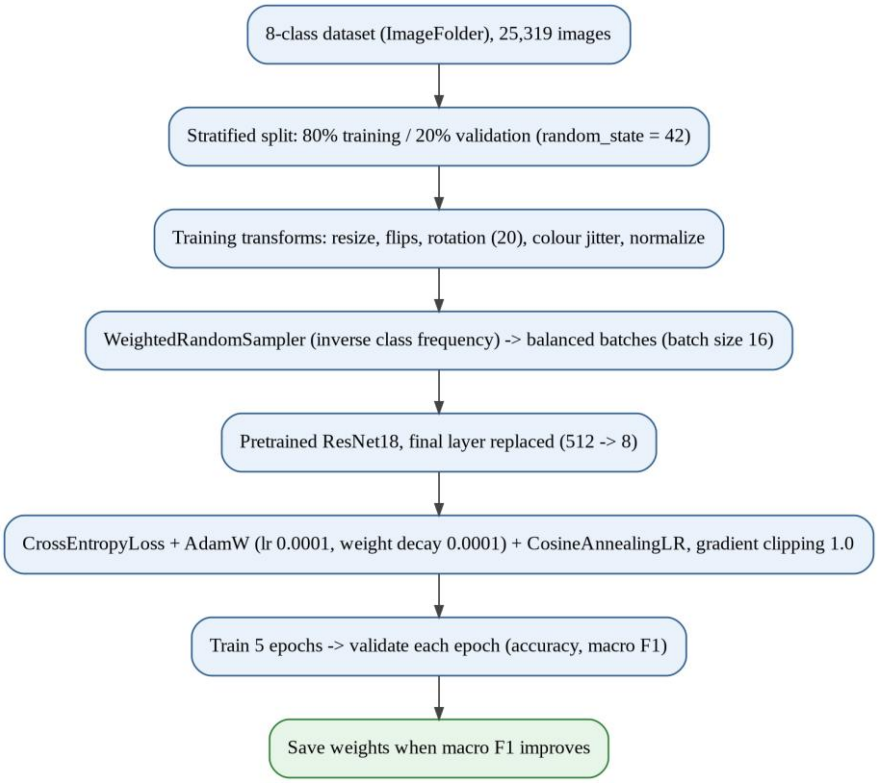


Figure 5.9 Model Training Workflow

Table 5.5 Training Configuration of the 8-Class Model

Parameter	Value
Base model	ResNet18 pretrained on ImageNet; final layer replaced (512 → 8)
Input size	224 × 224 pixels
Batch size	16
Epochs	5
Optimizer	AdamW (learning rate 0.0001, weight decay 0.0001)
Learning-rate schedule	Cosine annealing (T_max = 5)
Loss function	Cross-entropy loss
Class balancing	WeightedRandomSampler (inverse class frequency, with replacement)
Augmentation	Horizontal and vertical flip, rotation up to 20°, colour jitter (brightness 0.2, contrast 0.2, saturation 0.2, hue 0.05)
Gradient clipping	Maximum norm 1.0
Data split	80% training / 20% validation (stratified, random_state = 42)
Model selection	Weights saved when the validation macro F1-score improves

Training of the Stage 1 Model

The Normal Skin Detector is a ResNet18 whose final layer has two outputs (diseased and normal). It was trained for five epochs on 1,000 images (500 Normal and 500 Diseased), of which 800 images were used for training and 200 images for validation. It learns to separate healthy skin from skin that shows a disease, so that only suspicious images are passed to the 8-class disease model. The results of this model are given in Section 6.2.

5.8 MODULE DESCRIPTION

The system is divided into functional modules. Each module handles one part of the workflow, and together they provide the multi-attribute output of the project. The modules are: User Authentication & Authorization, Image Upload & Pre-processing, Normal Skin Detection (Stage 1), Disease Classification (Stage 2), Result Display, Treatment Recommendation, User Profile & Session Management, and Model Training & Evaluation.

6. User Authentication & Authorization Module

This module controls access to the system. A new user registers with a username, e-mail and password; the password is converted to a hash before it is stored. During login the entered password is checked against the stored hash and, if it is correct, the username is saved in the session and the user is taken to the upload page.

- Registration with duplicate username and e-mail checking.
- Login with hashed-password verification and session creation.

- Protection of the prediction, treatment and profile routes by redirecting to the login page.

2. Image Upload & Pre-processing Module

This module receives the skin image from the upload form, checks that a file was selected, cleans the file name with secure_filename() and saves the image in the uploads folder. The saved image is then converted to RGB and transformed into a 224 × 224 normalised tensor for the models.

3. Normal Skin Detection Module (Stage 1)

The Stage 1 ResNet18 model classifies the image as normal or diseased. If the skin is normal, the system returns "Normal Skin" with its confidence and does not run the disease model.

4. Disease Classification Module (Stage 2)

When the skin is not normal, the Stage 2 ResNet18 model classifies the image into one of the eight disease classes. The module returns the predicted class, its confidence and the Stage 1 confidence.

5. Result Display Module

This module shows the uploaded image together with the predicted label and the confidence percentage. For a diseased result it also shows a "View Treatment Recommendation" button and a warning that the system is educational. For normal skin the treatment button is hidden. A "Try Another Image" option lets the user repeat the process.

6. Treatment Recommendation Module

This module shows the detected condition and presents medicinal plant information (Aloe Vera, Turmeric, Neem and Green Tea) and general skin-care guidance (gentle skin care, sun protection, moisturisation and medical consultation). An important advisory states that herbal products must not be applied to suspicious or damaged skin without medical advice and that suspected skin cancer needs professional evaluation.

7. User Profile & Session Management Module

The profile page reads the username and e-mail of the logged-in user from the database and displays them. The logout route clears the session and returns the user to the home page.

8. Model Training & Evaluation Module

This offline module trains the ResNet18 disease classifier, evaluates it on the validation set using accuracy and macro F1-score, and saves the best weights and the class names file used by the web application.

5.9 INPUT DESIGN

Input design focuses on collecting correct data from the user through simple forms with validation on both the browser and the server.

Types of Inputs

1. **Registration Form:** Used by new users to create an account. Input fields: Username, E-mail, Password. Validation: all fields are required, the e-mail must be in a valid format, and the username and e-mail must be unique.
2. **Login Form:** Used to access the system. Input fields: Username, Password. Validation: both fields are required and the password is verified against the stored hash.
3. **Skin Image Upload Form:** Used to submit the image for analysis. Input field: image file (accepts image types only). Validation: a file must be selected and the file name is cleaned before saving.

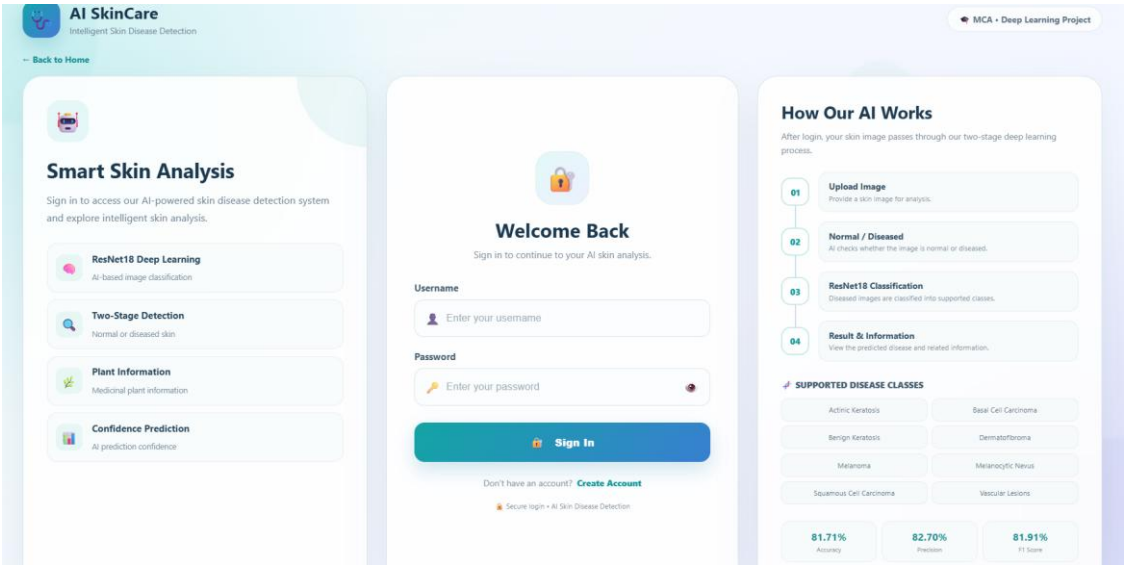


Figure 5.10 Login Page (Input Form)

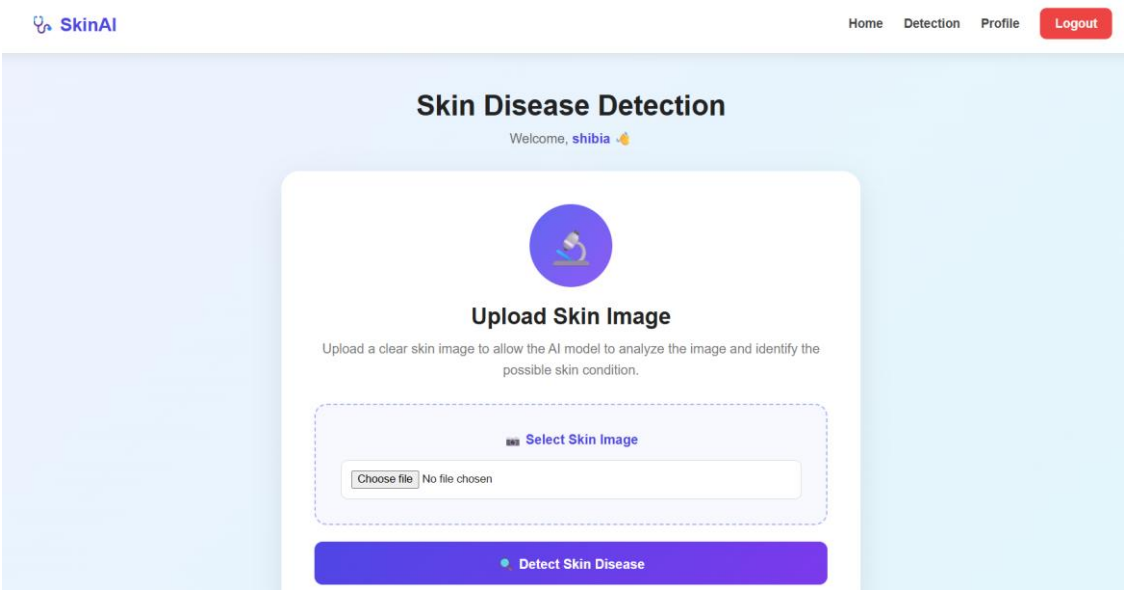


Figure 5.11 Skin Image Upload Page

5.10 OUTPUT DESIGN

Output design presents the results in a clear and understandable form. The outputs combine the multiple attributes produced by the system.

Examples of Outputs

1. **Prediction Result Page:** Shows the uploaded image, the prediction (Normal Skin or the disease code) and the confidence percentage. For diseased results a treatment button and a medical advisory are shown.
2. **Treatment Recommendation Page:** Shows the detected condition, medicinal plant information, general skin-care guidance and an important advisory.
3. **Profile Page:** Shows the username and e-mail of the logged-in user.

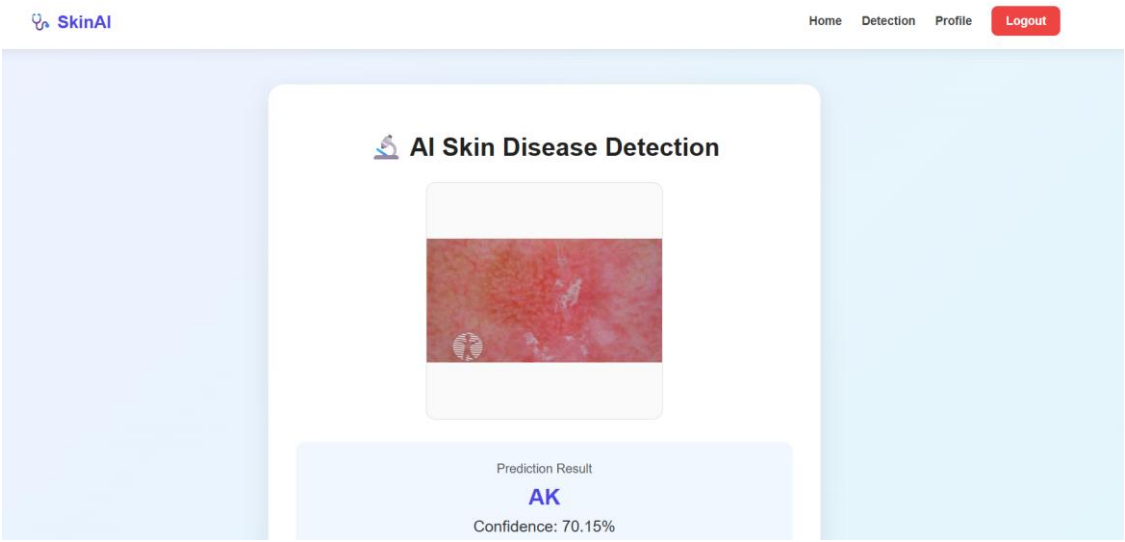


Figure 5.12 Prediction Result – Diseased Skin (AK)

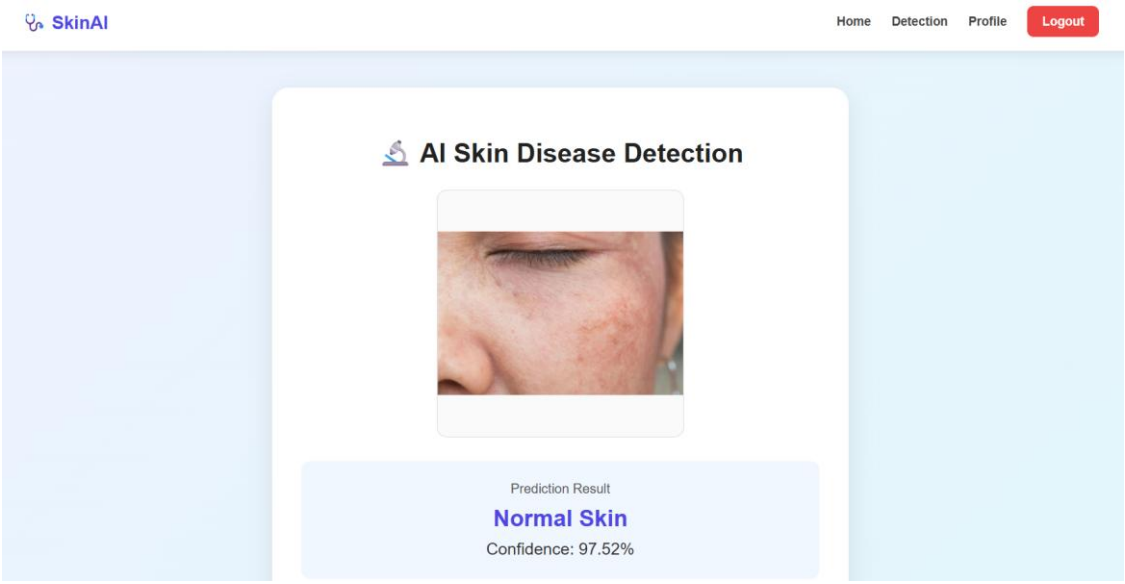


Figure 5.13 Prediction Result – Normal Skin

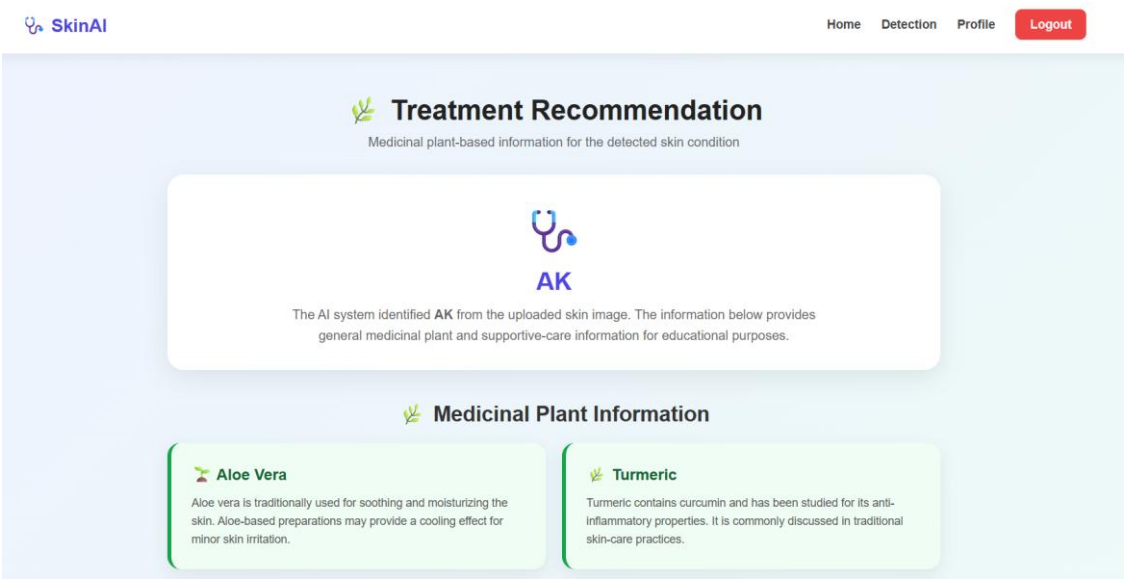


Figure 5.14 Treatment Recommendation Page

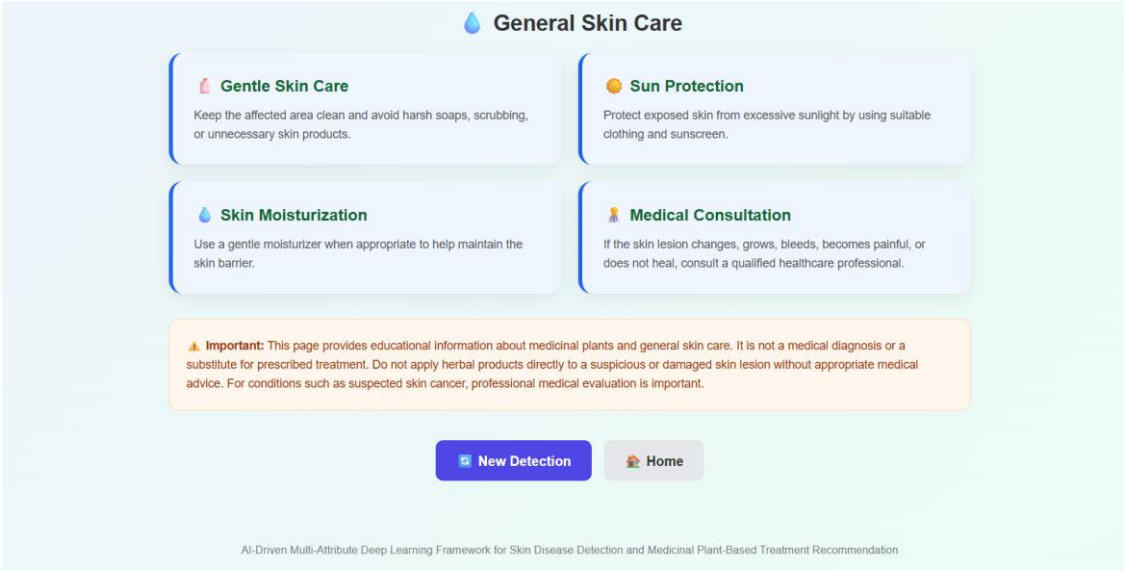


Figure 5.15 General Skin Care Section

CHAPTER 6

SYSTEM TESTING

SOFTWARE TESTING

The system was tested to confirm that the web application and the deep learning models work correctly. Functional testing covered registration, login, access control, image upload, normal and diseased prediction, the treatment page, the profile page and logout. Model testing measured how well the 8-class disease classifier performs on the validation images using accuracy, precision, recall, F1-score and a confusion matrix. The related screens are shown in Figures 5.10 to 5.15 and in the screenshots of Chapter 9.

6.1 TESTCASE

Table 6.1 LOGIN PAGE

Test Case ID	Test Description	Input	Expected Result	Status
TC01	Login with valid credentials	Registered username and password	Login successful; user is taken to the image upload page	Pass
TC02	Login with wrong password	Valid username, wrong password	“Invalid username or password.” Message is displayed	Pass
TC03	Login with unregistered username	Username that is not registered	“Invalid username or password.” Message is displayed	Pass
TC04	Login with empty fields	Blank username and password	Browser “required field” validation message is displayed	Pass

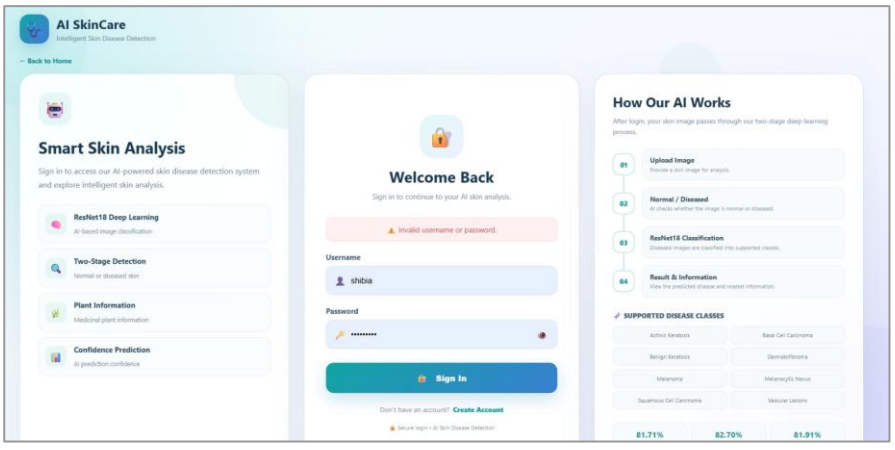


Figure 6.1 Test Case TC02 – Login with Wrong Password

Table 6.2 REGISTRATION PAGE

Test Case ID	Test Description	Input	Expected Result	Status
TC05	Register with valid data	Username, valid e-mail, password	Account created; user is redirected to the login page	Pass
TC06	Register with missing fields	Leave mandatory fields blank	Browser or server message "Please fill all fields."	Pass
TC07	Register with duplicate details	Already registered username or e-mail	"Username or email already exists." Message is displayed	Pass
TC08	Register with invalid e-mail format	E-mail without "@"	Browser e-mail format validation message is displayed	Pass

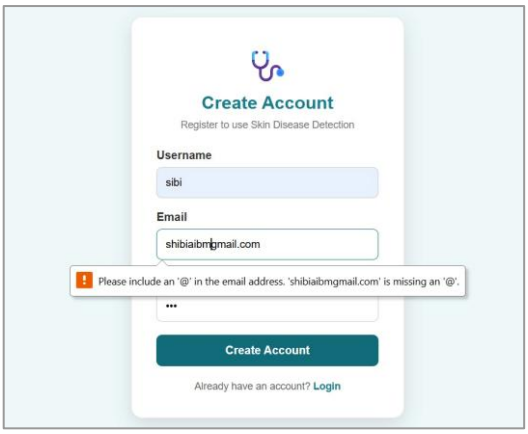


Figure 6.2 Test Case TC08 – Register with Invalid E-mail Format

Table 6.3 IMAGE UPLOAD AND PREDICTION

Test Case ID	Test Description	Input	Expected Result	Status
TC09	Open prediction page without login	Direct access to /predict	User is redirected to the login page	Pass
TC10	Submit without selecting an image	No file chosen	Browser validation message or "Please select an image."	Pass
TC11	Upload a normal skin image	Image of healthy skin	Result "Normal Skin" with confidence (for example 97.52%); treatment button not shown	Pass
TC12	Upload a diseased skin image	Image of a skin lesion	Disease class with confidence (for example AK 70.15%); treatment button shown	Pass
TC13	Display of uploaded image	Any valid image	Uploaded image is displayed above the prediction on the result page	Pass



Figure 6.3 Test Case TC10 - Submit without Selecting an Image

Table 6.4 TREATMENT, PROFILE AND SESSION

Test Case ID	Test Description	Input	Expected Result	Status
TC15	View treatment recommendation	Click "View Treatment Recommendation"	Treatment page shows plant information, skin-care guidance and the advisory	Pass
TC16	Open treatment page without login	Direct access to /treatment/AK	User is redirected to the login page	Pass
TC17	View profile	Click Profile after login	Username and e-mail of the user are displayed	Pass
TC18	Logout	Click Logout	Session is cleared and the home page is displayed	Pass
TC19	Try another image	Click "Try Another Image"	Upload page is displayed again	Pass

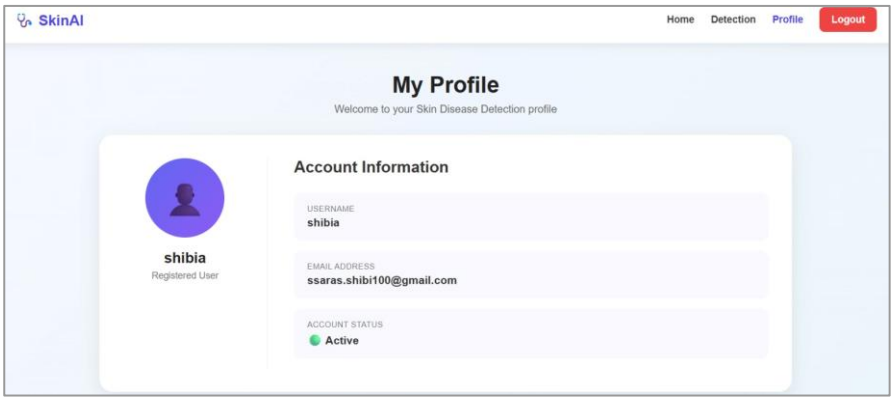


Figure 6.4 Test Case TC17 - View Profile

6.2 MODEL PERFORMANCE EVALUATION

The 8-class disease classifier was evaluated on a validation set of 5,067 images that were not used for training. The following measures were used.

- **Accuracy** = (number of correct predictions) / (total predictions).
- **Precision** = TP / (TP + FP): of the images predicted as a class, how many truly belong to it.
- **Recall** = TP / (TP + FN): of the images that truly belong to a class, how many were found.
- **F1-score** = 2 × Precision × Recall / (Precision + Recall).
- **Macro average** gives each class equal weight, while the **weighted average** weights each class by its number of images.

Out of 5,067 validation images, 4,140 were classified correctly. The overall results are given in Table 6.5.

Table 6.5 Overall Performance of the 8-Class Model

Metric	Value
Validation images	5,067
Correct predictions	4,140
Accuracy	81.71%
Weighted precision	82.70%
Weighted recall	81.71%
Weighted F1-score	81.91%
Macro precision	75.65%
Macro recall	86.27%
Macro F1-score	80.28%

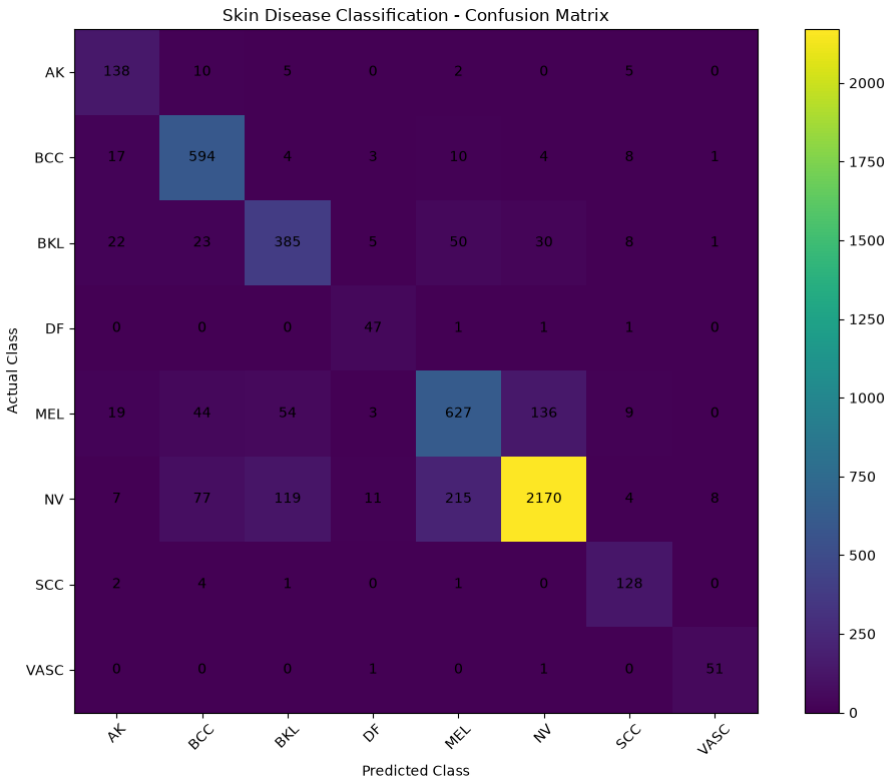


Figure 6.5 Confusion Matrix of the 8-Class Disease Model

The confusion matrix shows the actual class in each row and the predicted class in each column. The strong diagonal shows that most images are classified correctly. The class-wise precision, recall and F1-score calculated from the matrix are given in Table 6.6.

Table 6.6 Class-wise Performance Metrics

Class	Images	Precision (%)	Recall (%)	F1-score (%)
AK	160	67.3	86.2	75.6
BCC	641	79.0	92.7	85.3
BKL	524	67.8	73.5	70.5
DF	50	67.1	94.0	78.3
MEL	892	69.2	70.3	69.7
NV	2,611	92.7	83.1	87.6
SCC	136	78.5	94.1	85.6
VASC	53	83.6	96.2	89.5

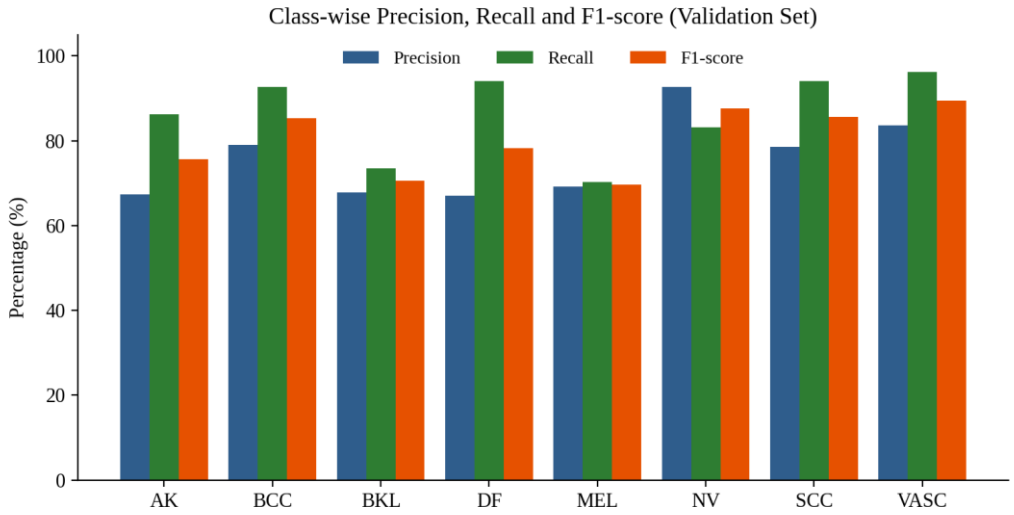


Figure 6.6 Class-wise Precision, Recall and F1-score

Analysis of Results

- The model reaches 81.71% accuracy on eight classes with a very uneven class distribution, which is a good result for a lightweight ResNet18 trained for only five epochs.
- Rare classes obtain high recall: DF 94.0%, SCC 94.1% and VASC 96.2%. This shows the benefit of the weighted sampler. The cost is lower precision for some of these classes, because some images from larger classes are predicted as rare classes. This is why macro recall (86.27%) is higher than macro precision (75.65%).
- Melanoma (MEL) and benign keratosis-like lesions (BKL) are the weakest classes, with F1-scores of 69.7% and 70.5%. Melanoma and nevus look visually similar, so they are confused with each other most often (Table 6.7).
- Melanocytic nevus (NV) obtains the highest precision (92.7%) because it has the largest number of training images.

Table 6.7 Most Frequent Misclassifications

Actual Class	Predicted As	Number of Images
NV	MEL	215
MEL	NV	136
NV	BKL	119
NV	BCC	77
MEL	BKL	54
BKL	MEL	50
MEL	BCC	44

Stage 1 Evaluation

The Normal Skin Detector was trained using 1,000 images, with 800 images for training and 200 images for validation. The model achieved a maximum validation accuracy of 100.00% in Epoch 3 and a final validation accuracy of 99.50% after 5 epochs, with a final training accuracy of 99.88%. The model classifies images into Normal and Diseased categories.

Table 6.8 Stage 1 Normal Skin Detector Evaluation

Parameter	Value
Total images used	1,000
Normal skin images	500
Diseased skin images	500
Training images	800
Validation images	200
Epochs	5
Maximum validation accuracy	100.00% (Epoch 3)
Final validation accuracy	99.50% (after 5 epochs)
Final training accuracy	99.88%
Output classes	Normal, Diseased

Because the Stage 1 validation set contains only 200 images, these figures show that normal and diseased skin are easy to separate in the available data, but they do not guarantee the same accuracy on every real-world photograph. The output is therefore always presented with its confidence and a medical advisory, and must be confirmed by a doctor, especially for melanoma, where wrong predictions can be serious.

CHAPTER 7

SYSTEM IMPLEMENTATION

7.1 INTRODUCTION

System implementation converts the designed modules into a working application: preparing the dataset, training the deep learning models, building the Flask web application, connecting the prediction module, testing the workflow and preparing the system for use.

7.2 IMPLEMENTATION PROCESS

1. Environment Setup

Python 3.9 or above was installed, and the required libraries (Flask, Werkzeug, PyTorch, torchvision, Pillow, scikit-learn and NumPy) were installed using pip. Visual Studio Code was used as the development environment.

2. Dataset Preparation

The skin lesion images were arranged in one folder per class inside dataset_8class so that they could be loaded with the torchvision ImageFolder class. Normal skin images were kept separately for the Stage 1 detector.

3. Model Training

The script train_resnet18_improved.py loads a pretrained ResNet18, replaces the final layer with an eight-output layer, applies augmentation, balances the classes with a weighted sampler and trains the network for five epochs with the AdamW optimiser. The best weights are saved as resnet18_skin_model_improved.pth and the class names to class_names.json. The Stage 1 normal/diseased detector (two-output ResNet18, 1,000 images) is stored as normal_skin_detector.pth.

4. Database Implementation

The SQLite database users.db and its users table are created automatically by init_db() when app.py starts. The table has a unique username and e-mail, and passwords are stored as hashes.

5. Web Application Development and Integration

prediction.py loads both models and the class names once when the server starts. The /predict route saves the uploaded image and calls predict_image(). The returned status, class and confidence are passed to result.html.

6. Testing and Running the Application

The functional test cases of Chapter 6 were executed for valid and invalid inputs, and the disease model was evaluated with the confusion matrix. The application is started with the command **python app.py** and opened at http://127.0.0.1:5000; for wider use it can be deployed on a Python-compatible web host with production settings.

CHAPTER 8

CONCLUSION AND FUTURE ENHANCEMENT

CONCLUSION

The project “Multi-Attribute Skin Disease Detection and Treatment Recommendations using Deep Learning” successfully combines deep learning with a web application to give users a quick, image-based skin screening. A two-stage ResNet18 pipeline first separates normal skin from diseased skin and then classifies eight skin disease classes. On the validation set of 5,067 images the disease model achieved 81.71% accuracy, 82.70% weighted precision and 81.91% weighted F1-score, with high recall for rare classes thanks to the balanced sampling strategy. The Stage 1 normal/diseased detector reached a final validation accuracy of 99.50%. The Flask application provides secure registration and login, image upload, prediction with confidence and a treatment page with medicinal plant information and general skin-care guidance.

The analysis also shows the limits of the system. Melanoma and nevus are still confused with each other, and the treatment page gives the same general information for every disease. The system is therefore an educational tool and an awareness aid, not a replacement for a dermatologist. Its main value is that it encourages users to seek medical advice early.

SCOPE FOR FUTURE ENHANCEMENT

1. **Disease-specific Treatment Information:** Provide different, medically reviewed recommendations for each of the eight classes instead of the same general plant list.
2. **Higher Accuracy:** Train for more epochs, use stronger architectures such as EfficientNet or Vision Transformers, and apply test-time augmentation to improve melanoma and BKL results.
3. **Larger and Better Normal-Skin Data:** Increase the number of normal skin images and evaluate the Stage 1 detector on independent clinical-style photographs.
4. **Explainable AI:** Show Grad-CAM heat maps so that users and doctors can see which part of the image influenced the prediction.
5. **Prediction History:** Store each user’s predictions in the database and show a history of past results and trends.
6. **Security Improvements:** Keep the secret key in an environment variable, disable debug mode in production, validate uploaded file types and sizes, and add CSRF protection.
7. **Mobile Application and Multilingual Support:** Provide an Android/iOS app with camera capture and local-language support.
8. **Doctor Consultation Integration:** Allow users to share the result with a dermatologist or book a teleconsultation.

CHAPTER 9

APPENDICES

9.1 SOURCE CODE

A. prediction.py – Two-Stage Prediction Module

```
import os
import json
import torch
import torch.nn as nn
from torchvision import models, transforms
from PIL import Image
# ================================================
# PATHS
# ================================================
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
# Stage 1: Normal / Diseased
NORMAL_MODEL_PATH = os.path.join(
    BASE_DIR,
    "normal_skin_detector.pth"
)
# Stage 2: 8 Disease Classification
DISEASE_MODEL_PATH = os.path.join(
    BASE_DIR,
    "resnet18_skin_model_improved.pth"
)
CLASS_NAMES_PATH = os.path.join(
    BASE_DIR,
    "class_names.json"
)
# ================================================
# DEVICE
# ================================================
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)
print("Device:", device)
# ================================================
# LOAD DISEASE CLASS NAMES
# ================================================
with open(CLASS_NAMES_PATH, "r") as f:
    disease_classes = json.load(f)
print("Disease Classes:", disease_classes)
# ================================================
# IMAGE TRANSFORMATION
# ================================================
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
```

```
        std=[0.229, 0.224, 0.225]
    )
])
# ================================================
# STAGE 1 MODEL
# NORMAL / DISEASED
# ================================================
print("\nLoading Normal Skin Detector...")
normal_model = models.resnet18(weights=None)
normal_model.fc = nn.Linear(
    normal_model.fc.in_features,
    2
)
normal_model.load_state_dict(
    torch.load(
        NORMAL_MODEL_PATH,
        map_location=device
    )
)
normal_model = normal_model.to(device)
normal_model.eval()
# IMPORTANT:
# ImageFolder gave:
# 0 = diseased
# 1 = normal
normal_classes = [
    "diseased",
    "normal"
]
# ================================================
# STAGE 2 MODEL
# 8 DISEASE CLASSIFICATION
# ================================================
print("Loading 8-Class Disease Model...")
disease_model = models.resnet18(weights=None)
disease_model.fc = nn.Linear(
    disease_model.fc.in_features,
    len(disease_classes)
)
disease_model.load_state_dict(
    torch.load(
        DISEASE_MODEL_PATH,
        map_location=device
    )
)
disease_model = disease_model.to(device)
disease_model.eval()
print("Both models loaded successfully!")
# ================================================
# PREDICTION FUNCTION
# ================================================
def predict_image(image_path):
    # ---------------------------------------------
```

```
# OPEN IMAGE
# ----------------------------------------------
image = Image.open(image_path).convert("RGB")
# ----------------------------------------------
# PREPROCESS IMAGE
# ----------------------------------------------
image_tensor = transform(image)
image_tensor = image_tensor.unsqueeze(0)
image_tensor = image_tensor.to(device)
# ================================================
# STAGE 1: NORMAL / DISEASED
# ================================================
with torch.no_grad():
    output = normal_model(image_tensor)
    probabilities = torch.softmax(
        output,
        dim=1
    )
    confidence, predicted = torch.max(
        probabilities,
        1
    )
stage1_class = normal_classes[
    predicted.item()
]
stage1_confidence = confidence.item() * 100
# ================================================
# IF NORMAL
# ================================================
if stage1_class == "normal":
    return {
        "status": "normal",
        "class": "Normal Skin",
        "confidence": stage1_confidence
    }
# ================================================
# IF DISEASED → STAGE 2
# ================================================
with torch.no_grad():
    output = disease_model(image_tensor)
    probabilities = torch.softmax(
        output,
        dim=1
    )
    confidence, predicted = torch.max(
        probabilities,
        1
    )
disease = disease_classes[
    predicted.item()
]
disease_confidence = confidence.item() * 100
return {
```

```
    "status": "diseased",
    "class": disease,
    "confidence": disease_confidence,
    "stage1_confidence": stage1_confidence
  }
```

B. app.py – Prediction Route

```
@app.route("/predict", methods=["GET", "POST"])
def predict():
  # Prediction requires login
  if "username" not in session:
    return redirect(
      url_for("login")
    )
  # ---------------------------------------------------
  # GET REQUEST
  # ---------------------------------------------------
  # When user opens /predict directly,
  # show the prediction page.
  if request.method == "GET":
    return render_template(
      "predict.html",
      username=session.get("username")
    )
  # ---------------------------------------------------
  # POST REQUEST
  # ---------------------------------------------------
  image = request.files.get(
    "image"
  )
  # Check whether image was selected
  if image is None or image.filename == "":
    return render_template(
      "predict.html",
      username=session.get("username"),
      error="Please select an image."
    )
  # Secure filename
  filename = secure_filename(
    image.filename
  )
  # Create image path
  image_path = os.path.join(
    app.config["UPLOAD_FOLDER"],
    filename
  )
  # Save uploaded image
  image.save(
    image_path
  )
  # ===================================================
  # TWO-STAGE PREDICTION
  # ===================================================
```

```
    result = predict_image(
        image_path
    )
    # ========================================================
    # STAGE 1 -> NORMAL SKIN
    # ========================================================
    if result["status"] == "normal":
        return render_template(
            "result.html",
            status="NORMAL",
            disease="Normal Skin",
            confidence=result["confidence"],
            # For normal skin, Stage 1 confidence
            # is also the main confidence
            stage1_confidence=result["confidence"],
            image_filename=filename,
            username=session.get("username")
        )
    # ========================================================
    # STAGE 2 -> DISEASE DETECTED
    # ========================================================
    return render_template(
        "result.html",
        status="DISEASED",
        disease=result["class"],
        confidence=result["confidence"],
        stage1_confidence=result["stage1_confidence"],
        image_filename=filename,
        username=session.get("username")
    )
```

C. train_resnet18_improved.py – Data Split, Balanced Sampling and Model Setup

```
# STRATIFIED TRAIN / VALIDATION SPLIT
# ============================================================
targets = train_full.targets
indices = list(range(len(targets)))
train_indices, val_indices = train_test_split(
    indices,
    test_size=0.20,
    random_state=42,
    stratify=targets
)
print("Training images:", len(train_indices))
print("Validation images:", len(val_indices))
# ============================================================
# CREATE SUBSETS
# ============================================================
train_dataset = Subset(
    train_full,
    train_indices
)
val_dataset = Subset(
    val_full,
```

```
    val_indices
)
# ============================================================
# BALANCED SAMPLING
# ============================================================
train_targets = [
    targets[i]
    for i in train_indices
]
class_counts = torch.bincount(
    torch.tensor(train_targets),
    minlength=len(class_names)
).float()
print("\nTraining class counts:")
for i, name in enumerate(class_names):
    print(
        f"{name}: {int(class_counts[i])}"
    )
# Calculate weight for each class
class_weights = 1.0 / class_counts
# Weight for each individual training sample
sample_weights = torch.tensor([
    class_weights[label]
    for label in train_targets
], dtype=torch.double)
sampler = WeightedRandomSampler(
    weights=sample_weights,
    num_samples=len(sample_weights),
    replacement=True
)
# ============================================================
# DATA LOADERS
# ============================================================
train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    sampler=sampler,
    num_workers=0
)
val_loader = DataLoader(
    val_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=0
)
# ============================================================
# RESNET18
# ============================================================
print("\nLoading pretrained ResNet18...")
model = models.resnet18(
    weights=models.ResNet18_Weights.DEFAULT
)
model.fc = nn.Linear(
```

```
    model.fc.in_features,
    len(class_names)
)
model = model.to(device)
# ============================================================
# LOSS
# ============================================================
# Use normal CrossEntropyLoss because
# WeightedRandomSampler already balances classes.
criterion = nn.CrossEntropyLoss()
# ============================================================
# OPTIMIZER
# ============================================================
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=LEARNING_RATE,
    weight_decay=0.0001
)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer,
    T_max=EPOCHS
)
# ============================================================
# SAVE CLASS NAMES
# ============================================================
with open(CLASS_NAMES_PATH, "w") as f:
    json.dump(class_names, f)
```

9.2 SCREENSHOTS

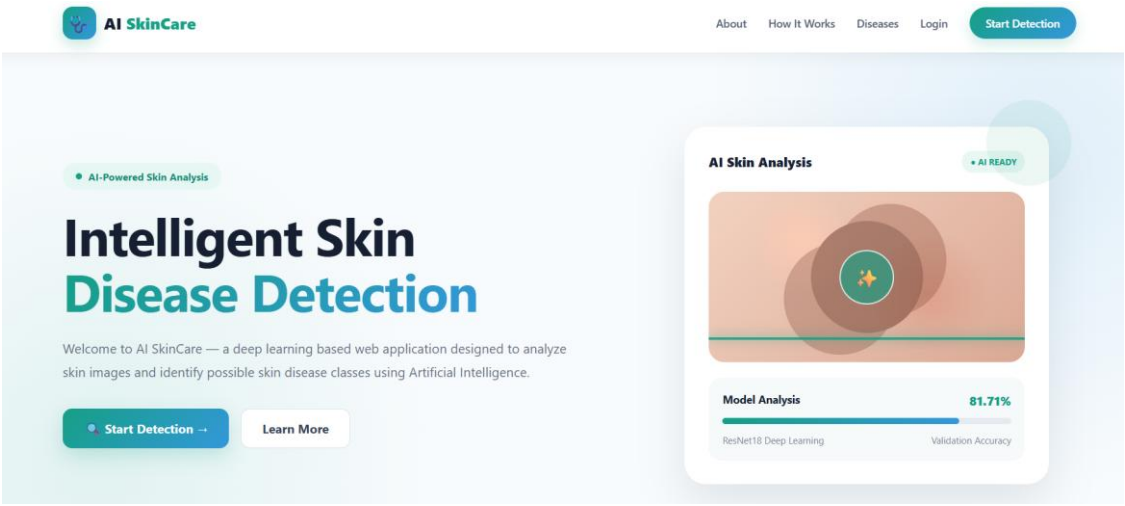


Figure 9.1 Home Page

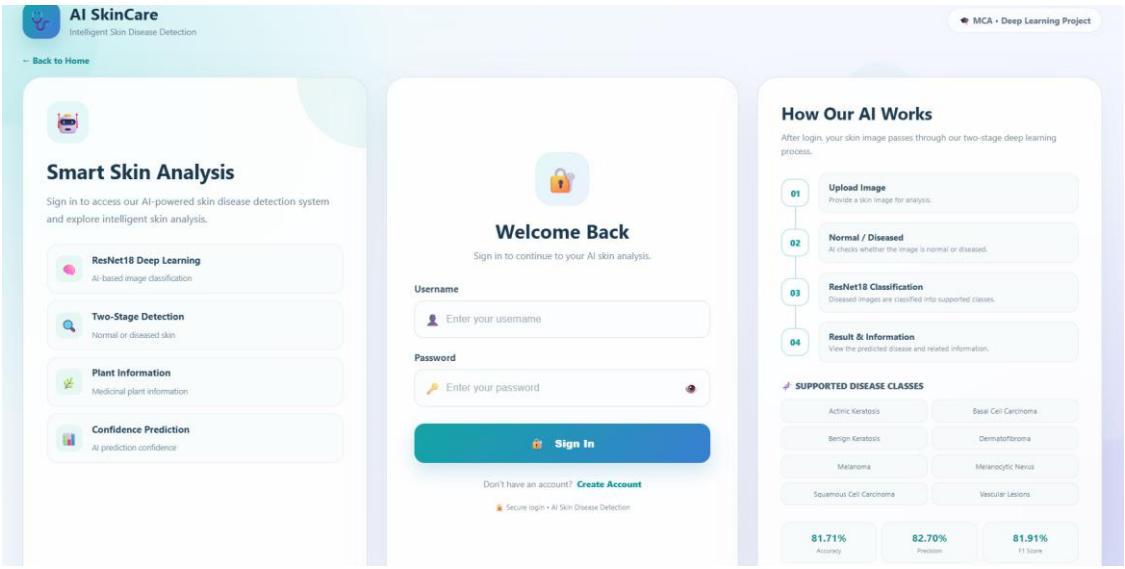


Figure 9.2 Login Page

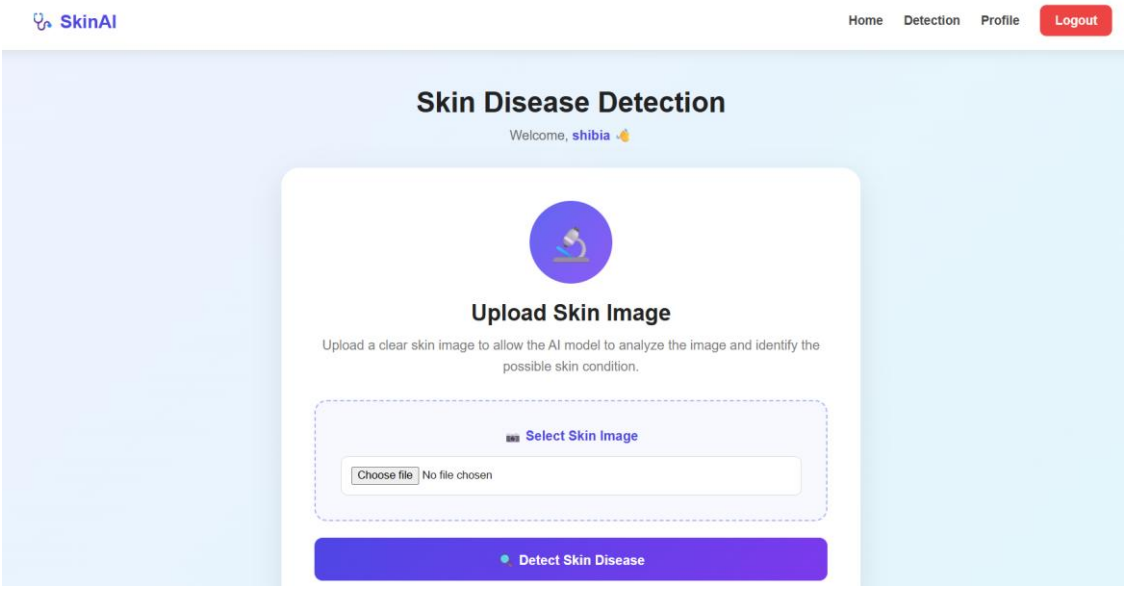


Figure 9.3 Skin Image Upload Page

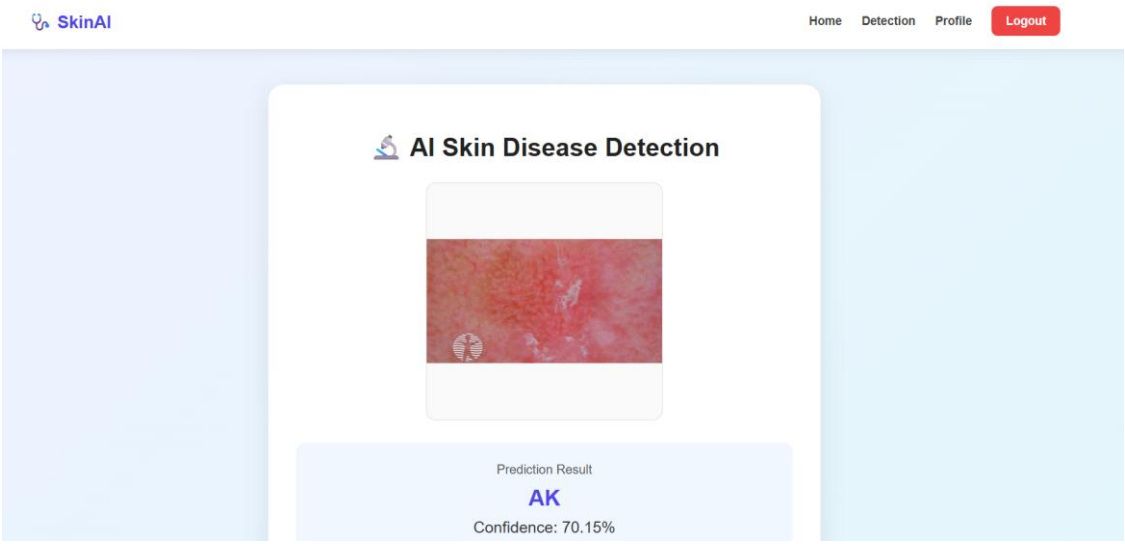


Figure 9.4 Prediction Result (AK)

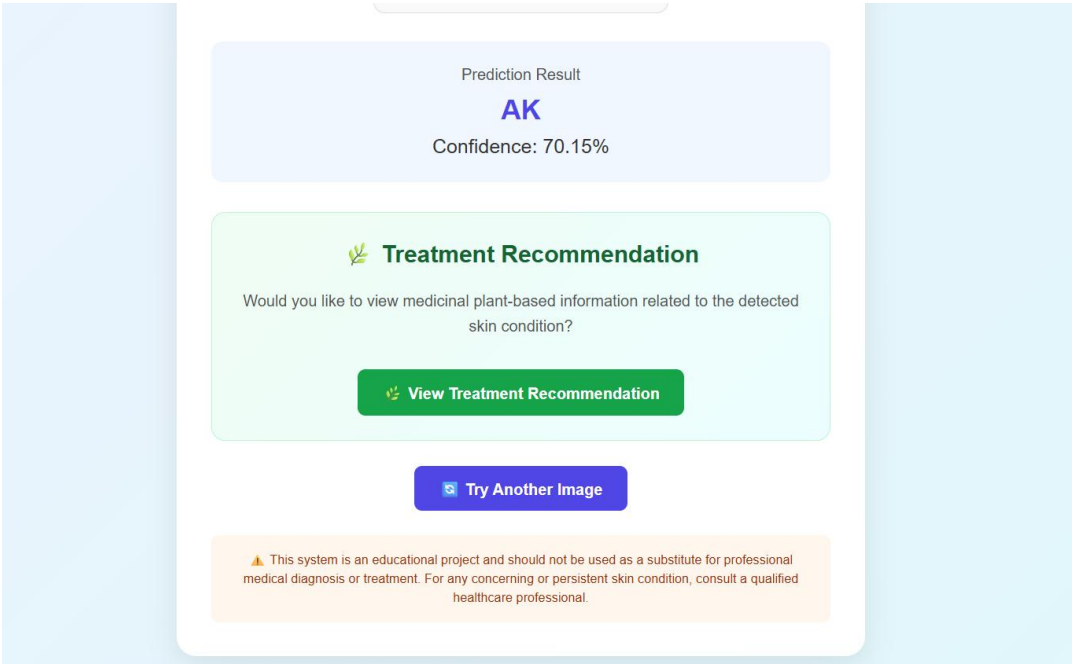


Figure 9.5 Treatment Recommendation Option

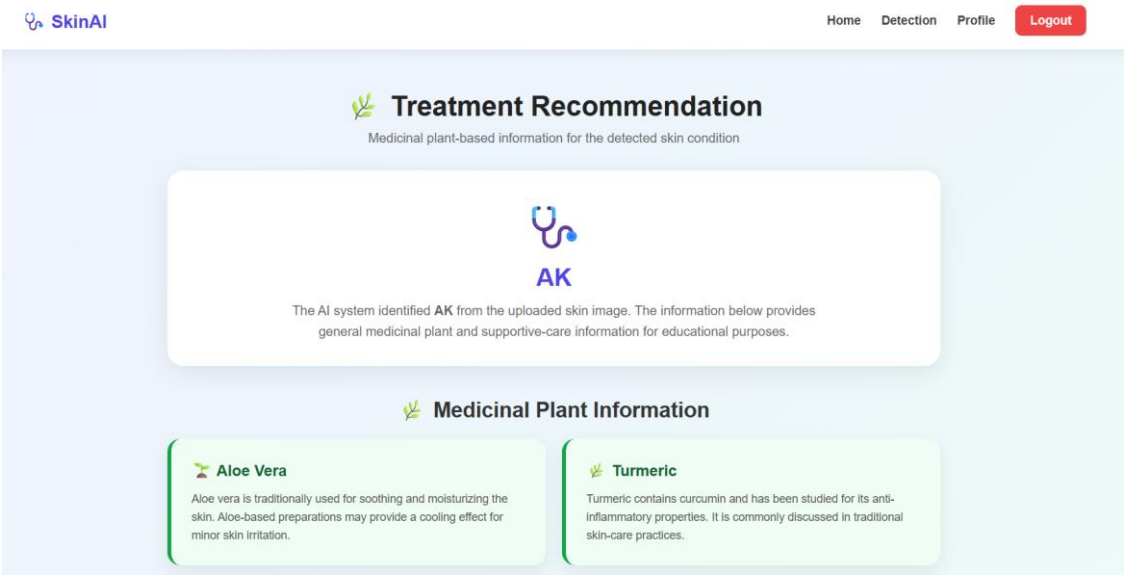


Figure 9.6 Treatment Recommendation Page

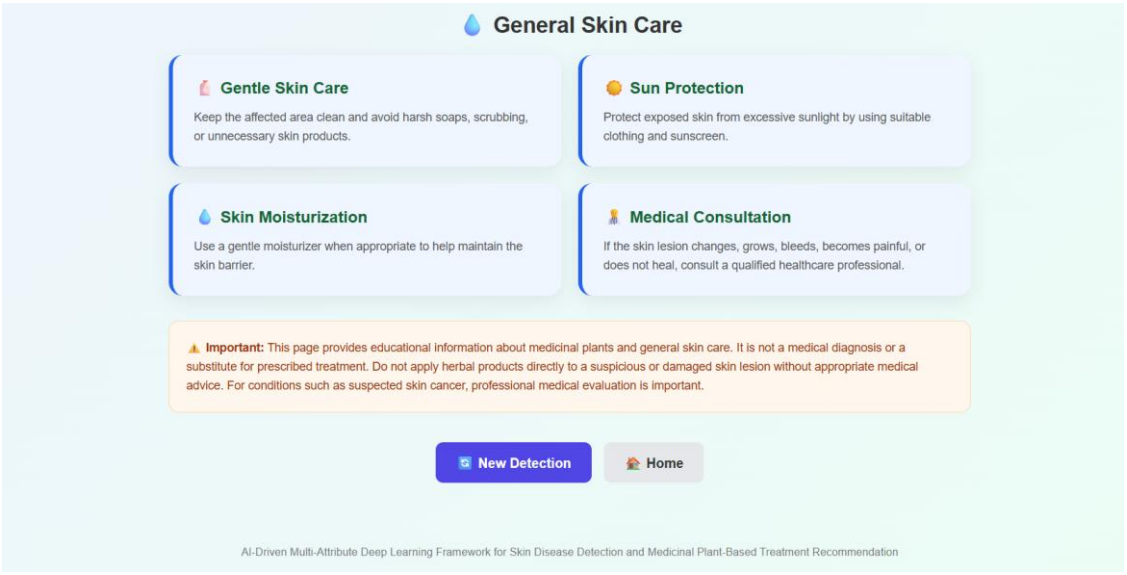


Figure 9.7 General Skin Care and Advisory

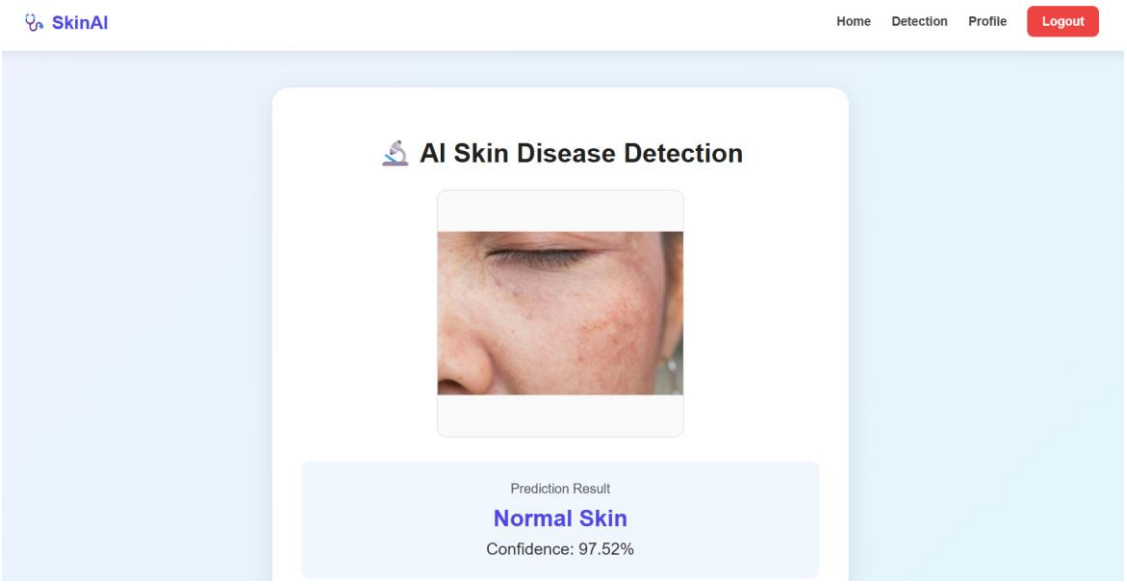


Figure 9.8 Prediction Result (Normal Skin)

CHAPTER 10

REFERENCES

Books and Research Papers:

1. K. He, X. Zhang, S. Ren and J. Sun, “Deep Residual Learning for Image Recognition,” Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778.
2. P. Tschandl, C. Rosendahl and H. Kittler, “The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions,” Scientific Data, vol. 5, Article 180161, 2018.
3. N. Codella et al., “Skin Lesion Analysis Toward Melanoma Detection 2018: A Challenge Hosted by the International Skin Imaging Collaboration (ISIC),” arXiv:1902.03368, 2019.
4. M. Combalia et al., “BCN20000: Dermoscopic Lesions in the Wild,” arXiv:1908.02288, 2019.
5. A. Esteva et al., “Dermatologist-level classification of skin cancer with deep neural networks,” Nature, vol. 542, pp. 115–118, 2017.
6. I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” International Conference on Learning Representations (ICLR), 2019.
7. A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” Advances in Neural Information Processing Systems (NeurIPS), 2019.

Web links referred:

https://challenge.isic-archive.com/

https://pytorch.org/docs/stable/index.html

https://pytorch.org/vision/stable/models.html

https://flask.palletsprojects.com/

https://scikit-learn.org/stable/

https://docs.python.org/3/library/sqlite3.html

https://developer.mozilla.org/en-US/docs/Web/HTML

https://developer.mozilla.org/en-US/docs/Web/CSS